

GJØVIK UNIVERSITY COLLEGE

BACHELOR THESIS

Snowman

A complete, effective and secure rule management system for Snort

Authors:
Thomas NYHEIM
Eirik SKOGSTAD
Eigil OBRESTAD

Supervisor:
Slobodan PETROVIC

May 11, 2014

Preface

This is the preface.

Acknowledgements

We would like to thank our mentor from Gjøvik University College, prof. Slobodan Petrovic, for his help and input during the project. We would also like to thank BKI and especially Jarle Kittilsen for assignig us this project and the help with forming requirements and the software testing.

This project is a rule management system for SNORT®[1], henceforth referred to as Snort.

[1] Snort® is a registered trademark of Sourcefire, Inc.

Glossary

This chapter contains an alphabetical list of technical terms used throughout the report along with their definition.

CLI Command-line interface.

Daemon A computer program that runs as a background process, usually without any user interaction.

DOM Document object model.

HDD Hard disk drive.

HTML Hypertext markup language.

HTTP Hypertext transfer protocol.

I/O Input / Output, used to describe data flow to and from devices such as harddrives.

IDS Intrusion detection system.

MD5 A cryptographic hash function producing a 128-bit hash value.

MVC Model-view-controller.

Open Source Software Software where the source code is publicly available and is licensed in such a way that the copyright holder grants the rights to study, alter and redistribute the software to anyone for any purpose [1].

RAM Random access memory.

RPC Remote procedure call.

RPM Rounds per minute.

SCP Secure Copy - a file transfer protocol.

Sensor A computer system wherein Snort and/or other security software is recording network traffic and produces alerts when malicious content is detected.

SSD Solid state drive.

SSL Secure sockets layer.

URL Uniform resource locator.

XML Extensible markup language.

Contents

Preface . . . . . **i**
Glossary . . . . . **ii**
Contents . . . . . **iii**
List of Figures . . . . . **vii**
List of Tables . . . . . **viii**
1 INTRODUCTION **1**
1.1 Project Background 1
1.2 Previous Work 1
1.3 Project Description 2
1.4 Target Audence 2
1.5 Project Objectives 3
1.5.1 Result Objectives 3
1.5.2 Effect Objectives 4
1.6 Academic Background 4
1.7 Framework 4
1.7.1 Software Development Methodology 4
1.7.2 Schedule 5
1.7.3 Project Organization 6
1.8 Document Structure 6
1.8.1 Special styles used 6
2 EXISTING SOLUTIONS **7**
2.1 Functionality comparison 7
2.2 Bring Home The Bacon 7
2.3 Snortmanager 9
2.4 Other notable programs 9
2.5 Research on Software Performance 9
3 REQUIREMENTS SPECIFICATION **10**
3.1 Functional Requirements 10
3.1.1 Program Workflow 10
3.1.2 Use Case Diagram 12
3.1.3 Domain Model 13
3.2 Supplemental Requirements 13
3.2.1 Functionality 13
3.2.2 Usability 13
3.2.3 Reliability 13
3.2.4 Performance 14
3.2.5 Security 14
3.2.6 Interoperability 14
3.2.7 Licensing 14
3.3 Constraints 14

3.3.1 Platform . . . 14
3.3.2 Data security . . . 14
3.3.3 Graphical User Interface . . . 15
3.4 Security Assessment . . . 15
4 CONCEPTS OF SNORT . . . 17
4.1 Rules . . . 17
4.2 Rule sets . . . 17
4.3 Generators . . . 17
4.4 References . . . 17
4.5 Event and detection filters . . . 17
5 DESIGN . . . 19
5.1 General Design . . . 19
5.1.1 Centralized or distributed setup? . . . 19
5.2 Server design . . . 20
5.2.1 Architecture . . . 20
5.2.2 User interface . . . 21
5.2.3 Database . . . 22
5.3 Client . . . 23
5.3.1 Architecture . . . 23
5.3.2 Database . . . 23
6 IMPLEMENTATION . . . 27
6.1 Programming languages . . . 27
6.1.1 Standards and guidelines . . . 28
6.2 Development environments . . . 29
6.2.1 Environments . . . 29
6.2.2 Version control . . . 29
6.3 Frameworks . . . 29
6.3.1 Django . . . 29
6.3.2 SQLAlchemy . . . 29
6.3.3 jQuery . . . 29
6.3.4 Bootstrap . . . 30
6.4 Core module . . . 30
6.4.1 Rules and rule revisions . . . 30
6.4.2 Rule sets . . . 31
6.4.3 Sensors . . . 31
6.4.4 Comments . . . 31
6.5 Update module . . . 31
6.5.1 Sources . . . 31
6.5.2 Files . . . 32
6.5.3 Running an update . . . 32
6.5.4 Rule parsing . . . 33
6.5.5 Classification, Generator and Reference Type parsing . . . 34
6.5.6 Saving . . . 34
6.5.7 Logging . . . 35
6.6 Distribute . . . 35
6.6.1 Initiating the distribute . . . 35

6.6.2 Synchronization . . . 35
6.6.3 Generating snort configuration . . . 36
6.7 Graphical User Interface . . . 36
6.7.1 Structure . . . 36
6.7.2 Design . . . 36
6.7.3 Overview of central GUI features . . . 37
6.8 Logging . . . 41
6.9 Configuration files . . . 42
6.9.1 Server configuration file . . . 42
6.9.2 Client configuration file . . . 42
6.10 Documentation . . . 43
6.11 Deployment . . . 43
6.11.1 Snowman server . . . 43
6.11.2 Snowman client . . . 45
7 SOFTWARE TESTING . . . 46
7.1 Strategy . . . 46
7.2 Description of Problems . . . 46
7.2.1 Rule insertion . . . 46
7.2.2 List Pagination . . . 47
7.2.3 The Ever Changing Number of Changes . . . 47
8 PERFORMANCE ANALYSIS . . . 48
8.1 Theoretical Analysis . . . 48
8.1.1 Effectiveness of Update Processing . . . 48
8.1.2 Effectiveness of Rule Distribution . . . 48
8.1.3 GUI Latency . . . 49
8.1.4 Linearity in Performance . . . 49
8.2 Practical analysis . . . 49
8.2.1 Definitions . . . 49
8.2.2 Environment . . . 49
8.2.3 Test suites . . . 49
8.2.4 Update processing tests . . . 50
8.2.5 Rule distribution tests . . . 52
8.2.6 Interface loading tests . . . 52
8.2.7 Abandoned tests . . . 52
8.3 Optimization . . . 53
9 RESULTS OF PERFORMANCE TESTING . . . 54
9.1 Update processing . . . 54
10 ANALYSIS OF PERFORMANCE TESTING RESULTS . . . 58
10.1 Effectiveness of update processing . . . 58
10.2 Effectiveness of rule distribution . . . 59
10.3 GUI latency . . . 59
10.4 Linearity in performance . . . 60
10.5 Conclusions . . . 60
11 CONCLUSIONS . . . 61
11.1 Results . . . 61
11.2 Future Development . . . 61

11.3 Group Evaluation 62
11.3.1 Introduction 62
11.3.2 Organization 62
11.3.3 Work distribution 63
11.3.4 What could have been done differently 63
11.3.5 Subjective views 63
11.4 Conclusion 63
Bibliography **64**

List of Figures

1 Development Model 5
2 Program Workflow 10
3 Program Modules 11
4 Use Case Diagram 12
5 Conceptual Class Diagram 13
6 Flow 18
7 General architecture 19
8 Server Architecture 21
9 Core server-database 23
10 Tuning server-database 24
11 Update server-database 25
12 Client Architecture 25
13 Client Database 26
14 GUI Structure 37
15 Rules page 37
16 Rulesets page 39
17 Update progressbar 39
18 Update changes 40
19 Form validation 41

List of Tables

1 Functionality comparison chart. 8
2 Security Assessment 16
3 Effects of different filter types. 18
4 Rule options used in this system. 32
5 Test systems. 50
6 Test cases in update processing. 51
7 Test results for update processing. 55
8 Test results for sensor synchronization. 56
9 Test results for interface performance. 57
10 Average processing time per rule. 58
11 Comparison of processing time for many files vs one file. 59
12 Empirical linear correlation coefficients for update processing. 60

1 INTRODUCTION

1.1 Project Background

The Norwegian Armed Forces Cyber Defense (CYFOR) unit for Computer Network Defense (CND), Avdeling BKI, is responsible for detecting and stopping cyber-attacks against the critical infrastructure and command and control systems utilized by the Norwegian military. One of the tools used to carry out this task is the open source IDS Snort [2].

Snort is a leading (ref?) open source IDS that is used to detect potentially unwanted or malicious network traffic based on predefined patterns described in signatures. There are both free and commercial signatures available for Snort and as Snort has been highly adopted by the security community, these signatures are continually updated and maintained. For large networks, an IDS will often operate in multiple parts of the network simultaneously, where each separate instance of the IDS is known as a *sensor*. The management of Snort signatures (henceforth referred to as rules) is cumbersome in multi-sensor environments without a centralized service which can control that all sensors are synchronized with the latest rules. Avdeling BKI has therefore tasked a group of students from Gjøvik University College (GUC) to design and develop a more functional, effective and secure rule management system for Snort that can continue to live on beyond this project.

1.2 Previous Work

Research [3] has shown that a good rule management system for Snort will improve the workflow and efficiency of the IDS significantly, and a prototype for such a system was built in 2012 called Bring Home The Bacon [3] (henceforth referred to as BHTB). This prototype has been in use at Avdeling BKI since 2012, but is suffering from lacks in functionality and efficiency due to being a proof-of-concept prototype. BHTB has, however, proven that there is a significant gain from having an intuitive Graphical User Interface (GUI) for managing rules. A GUI facilitates the work of the operator, as work is shifted from complex operations in a CLI to automatic execution of common routines triggered by buttons.

There was another similar project conducted at Gjøvik University College in 2012 called Snortmanager [4] which also deals with management of Snort rules. The Snortmanager project files are available as open source software[1], but no usage instructions are available. The source code has not been maintained since its initial release and is designed for a specific company in mind. Enabling, disabling and configuring rules is also cumbersome in this system as it is based on plaintext arguments in "policies" that users must construct themselves [4]. The rule distribution is also done manually from the files generated. All this makes Snortmanager unsuitable for Avdeling BKI and their routines.

Several add-ons and other Snort-related software exist[2], and a few of these are built

[1] http://github.com/stianja/SnortManager
[2] http://www.snort.org/snort-downloads/additional-downloads

to manage Snort rules, most notably Pulled_Pork[3] and Oinkmaster[4]. The key differences between all software mentioned in this section and Snowman will be further explained in chapter 2.

1.3 Project Description

This project is broken down to four main objectives:

1. Identify the challenges of rule management in IDS's such as Snort.
2. Describe how a good rule management system should operate.
3. Develop a good open source rule management system for Snort.
4. Write documentation for installation and usage of the system.

handling multiple individually configured sensors.

In broad strokes, this project has been about designing a system that provides a simple and efficient way of handeling rule management for the Snort IDS. The project is the spiritual successor to BHTB. Essentially taking good ideas from BHTB, adding a few of our own, and creating a more adoptable open source rule management system for Snort.

We utilized the latest stable version of Python as our programming language. C++ was also considered, but due to the limited time at our disposal, we felt we could create more functionality with Python, due to it requiring less memory management and the general "programmer friendly" ways of doing things.

The system required a database to function efficiently and the project primarily utilizes MySQL, as it is the de facto standard for the Linux platform. To make our database operations simpler, we've utilized a framework called Django[5] which offers us an abstraction layer on top of the database, so that we primarily only work with objects.

Furthermore, the project also consisted of creating a GUI that is user friendly and responsive.

The software that was created in this project, though still lacking some features and contains some minor bugs, largely fulfils the project goals outlined in section 1.5. We did not fulfil the goal of producing software that has reduced latency and delays, but we are confident that this is because we have sacrificed efficiency for functionality. We also feel that the functionality gained outweighs the efficiency lost and the efficiency curve is still very much within the linearity objective, as you can read about in chapter **??**.

Because the project has been so performance oriented, the report has evolved to also contain a small study into software performance. We've discovered that it is quite difficult to achieve performance in software when the expectations of functionality is high. We've also discovered the importance of conducting software performance testing before it is released, rather than after. Because we conducted a performance test on our software, we were able to implement optimizations that drastically improved the performance of one of our big operations within the software, the rule updating process.

1.4 Target Audence

Snowman is targeted at users of the Snort IDS in a larger production environment with many sensors and networks. More specifically, the target audience consists of security

[3] http://code.google.com/p/pulledpork
[4] http://oinkmaster.sourceforge.net

experts with a high degree of knowledge of Snort and computers in general.

This report is targeted at those that may be interested in discovering just how complex the rule structure of Snort really is, and academics that are interested in reading about software performance and how complexity plays into it.

1.5 Project Objectives

1.5.1 Result Objectives

We reiterate from the project plan[link to project plan] that this project has a goal of researching, designing and developing a system that as a minimum:

- Is more efficient than BHTB.
- Is developed as an engine that responds to API calls.
- Is modular in design and allows future changes and additions to not affect existing functionality.
- Is scalable enough to handle both small and large sets of IDS sensors and rules in a production environment.
- Is able to download rules and rule sets from multiple customizable sources.
- Is able to distribute different rules to different IDS sensors.
- Can download and distribute rules automatically at set intervals.
- Is capable of exporting rules for sharing.
- Has the capability to activate, deactivate, filter and suppress rules on a per sensor basis, in a way that is user friendly.
- Supports multi-user interaction.
- Can efficiently and safely work with different installations of Snort and various other third party tools, such as SNORBY [6].
- Is easy to install.
- Has an internal logging system.
- Is well documented and can be easily adopted by anyone as open source software.

In addition, the project will attempt to add the following functionality if time permits:

- A graphical user interface.
- Said user interface must be user friendly and intuitive, and require only limited knowledge of Snort rules from the user.
- The system can also distribute rules to Suricata [7], another Snort-like IDS, which can operate with the exact same rules as Snort.
- Implement security features that ensure confidentiality of rules.

- The project will investigate and potentially implement multi-threading technology or other optimization techniques to exploit modern hardware capacities.
- The system is capable of validating and checking rules for errors before they are distributed to sensors.
- The system supports commenting of the rules.
- The system supports writing custom rules and editing existing rules.

Once the system has been developed, it will be released as an Open Source Software to the public in a well-documented format.

1.5.2 Effect Objectives

Forming baselines from the current prototype BHTB and from Snortmanager, the project has a goal to further increase the efficiency of rule management by investigating and implementing programming techniques such as multi-threading and caching to reduce latency and delays in the system by as much as 50%. The software must also maintain linearity in performance, i.e. 10 executions of the same job should take 10 times, or less, the amount of time as a single execution.

The achievement of these goals is to be measured by conducting several performance tests on the final software solution. Results will also be compared to the performance of BHTB and Snortmanager.

1.6 Academic Background

This project has probably benefited greatly from the fact that all three project members are all taking different computer science courses. One member has studied computer engineering, the second has been learning the operations trade while the third has been studying information security. This has given us a unique opportunity to draw from each others strengths and approach the project from different perspectives. We have also benefitted from the diverse prior experience in the group, two of the members spent the year prior to this project at an internship at CERN in Switzerland. The third member of the team has worked for Avdeling BKI for the past 7 years prior to this project and has good knowledge of what the client needs and experience from the information security field in general.

1.7 Framework

1.7.1 Software Development Methodology

As this project has been very time restricted, the development needed be as efficient as possible. We therefore had to find a model that utilized the manpower we had available in the best possible way. The size of the group was quite small, so big and complex development models would add too much bureaucracy to coordinate the project and was thus overkill.

As we wanted to have a lightweight development model, some form of an agile[8] development model with a set of development cycles was preferred. This way, each group member could take an active part in the project and all members could feel a sense of ownership.

Developing iterative and incrementally, we got the best from both the agile development world and from the very well defined waterfall approach[9]. We were able to

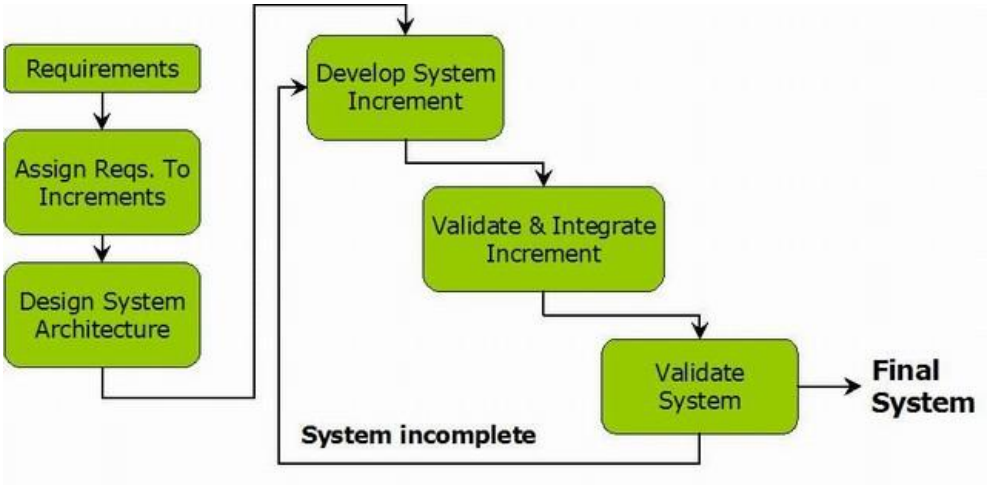


Figure 1: Development Model

get feedback throughout the project, and have a structured and reasonable way of actually doing changes to the plan while the project was running. At the same time we could be able to maintain the overall plan within cycles, so that it was possible to work individually, while still being well coordinated.

To be able to develop all the modules of the project within the timeframe, each development cycle consisted of multiple modules or functionality developed in parallel. This allowed us to maximize our utilization of time and personnel.

The start of each cycle consisted of a meeting where we defined the goals for that cycle and identifying the individual tasks that we needed to get done. This is similar to the start of a sprint in the development model Scrum[10].

In the development part of the cycle, we worked individually on the tasks that were defined in the planning. Regular meetings and cooperative programming within the group ensured that we all pulled the project in the same direction.

At the end of each cycle we would have a working set of software, which ideally complied with all the goals set in the start of each cycle. We held meetings with Avdeling BKI between each cycle to present the progress, and to let them influence the direction of the project.

After all the development cycles had been completed, we then made a final assurances that everything worked as intended and that all modules coexisted in harmony. This part of the development also consisted of some hands-on testing by the client and performance testing.

1.7.2 Schedule

We planned this project to have a fairly comprehensive initial phase for mapping and designing of the system. We spent the better part of the first two weeks figuring out how to implement the system. After the two weeks and before we started actually programming, we held a “project decision meeting" with the client to confirm that we had modelled the system correctly, to get feedback and to get the final go-ahead.

We then began the implementation phase which consisted of development cycles as described above in section 1.7.1. This phase was devided into five parts, consisting of

four development cycles and a final cleanup/testing cycle. All the development cycles lasted two weeks, aside from the last development cycle and the cleanup/testing cycle, which lasted one week each.

During those cycles we conducted meetings with both our mentor and our client every two weeks, timed so that these meetings took place on alternating weeks. This ensured that we got continual feedback and advice throughout the entire process.

After these weeks, we completely shifted focus from the software to the project report and spent the final 5-6 weeks working on that. Because we had done a thorough job during the first few weeks of the project, we had a lot of the contents for the report more or less created.

1.7.3 Project Organization

We organized the project work a little differently throughout the project, reflecting on the fact that we had multiple phases. During the first few weeks we spent most of the time together in a group room, working together and discussing things. Once most things had been worked out and the implementation phase began, there was more room and need to work individually to produce code. We still gathered together every so often to syncronize our progress and plan the next course of action.

Throughout the project, most decisions have been met either by confering with the client or achieving consensus within the group.

1.8 Document Structure

TBC: How have we organized this document.

1.8.1 Special styles used

Software names are written in a different font throughout the report, like so: Snowman. Source code is listed in an enclosed box with syntax highlighting (listing 1.1). Source code can also appear in the text with the same syntax highlighting as the listings, especially note classnames which will appear as so: `ClassName`.

```
1 def printHello():
2   # Print a message
3   print 'Hello World!'
```

Listing 1.1: Source code example.

2 EXISTING SOLUTIONS

To identify exactly what Snowman does differently than other, similar software, we will quickly go over their functionality and how they differ.

2.1 Functionality comparison

To easily visualize how the various software is different from each other, we have gathered key functionality from all of them and created a list of support for them.

***putt inn tabell her ***

2.2 Bring Home The Bacon

Bring Home The Bacon was a by-product prototype of the Master thesis written by Henriksen in 2012 [3]. In the thesis, Henriksen seeks to prove that having a central, graphical-oriented, system to administer multiple Snort-sensors and their rules results in a signficant improvement in user workflow over the more traditional command-line environment. Henriksens tests show that such a system can double the users' work efficiency, if not more, as the command-line environment would scale very badly as the number of sensors increases. Henriksen also outlines a rule management process, which has been helpful during the initial phases of this project to map the program workflow (section 3.1.1).

Despite being a prototype, BHTB offers the essential functionality required for managing rule sets on multiple sensors. It can display a list of rules which can also be turned on and off on different sensors. It can also display a list of rule sets that can also be turned on and off, which will effect all rules in the rule set. This is considered a drawback, as BHTB does not offer a mechanism where rule sets can be disabled without affecting the state of the rules within. This means that the original state of the rules in a rule set is lost whenever the state of the rule set is changed. Snowman improves this by keeping the state of the rule sets separate from the state of the rules.

BHTB distributes rules to the sensors by generating all files centrally and sending them one by one via SCP. It determines which rules to distribute by comparing the main rule table with a second table representing which rules the sensor should not have. This way of distributing rules is considered ineffective, and has been solved differently in Snowman (section 5.1). BHTB also supports manual creation of rule thresholds and suppressions, but does not offer any input validation, which makes the sensors vulnerable to crashes[1] in the case of invalid input. This weakness is taken into consideration in Snowman, which offers a comprehensive input validation. BHTB is programmed with Python and uses the framework Tornado [11] as its Web-server, which works similar to Django, a framework used by Snowman, with it's URL API system (section **??**). It also uses SQLite [12] as its database, which has limited abilities when it comes to relations and more complex data structures.

According to Avdeling BKI, BHTB really begins to show its limitations once the system

[1] Snort will simply not restart if something in the configuration files is malformed.

Feature	Snowman	BHTB	Snortmanager	Pulled_Pork	Oinkmaster
Rule download and storage	✓	✓	✓	✓	✓
Sensor administration	✓	✓	✓		
Graphical user interface	✓	✓	✓		
Automatic updates	✓	✓		✓	✓
Automatic synchronisation	✓	✓		✓	✓
Enable/disable rules	✓	✓			✓
Multiple sensors supported	✓	✓			✓
Can syncronize rules to sensors	✓	✓			✓
Displays change history of updates	✓	✓			✓
Can store meta data about rules	✓	✓			
Can turn on or off whole rulesets	✓	✓			
Can turn on or off rulesets on individual sensors	✓	✓			
Can add filters and suppressions to rules	✓	✓			
Can add filters and suppressions to rules on a per sensor basis	✓	✓			
Has an internal logging system	✓	✓			
Can display list of rules in the system	✓	✓			
Can display meta data in the system	✓	✓			
Normalized database	✓		✓	N/A	N/A
Does not require root to interact with Snort.	✓				
Turning rulesets on or off is non-propagating	✓		N/A	N/A	N/A
Can import filters and suppression from rule files	✓		✓	✓	✓
Has input validation	✓			N/A	N/A
Supports multiple database technologies	✓			N/A	N/A
Handles multi line rules	✓				✓
Can skip certain files	✓				✓
User can approve changes before sensors are updated	✓				✓
Is easy to install	✓				
Can import rules from arbitrary sources	✓				
Can organize rulesets in a hirarchy	✓				
Can create new custom rulesets	✓				
Can organize sensors in a hirarchy	✓				
Can syncronize rules to multiple	✓				

is populated by a lot of rules[2] and sensors, which makes it clear that it has a scalability problem. Avdeling BKI has used BHTB in a production environment since 2012 and experiences the software as slow and cumbersome.

2.3 Snortmanager

Beskriv hva den gjør

2.4 Other notable programs

There are two other pieces of software that also should be mentioned; Pulled_Pork and Oinkmaster are both programs that can download rules and update Snort-sensors. These programs also contain some of the features found in Snowman and BHTB, and Pulled_Pork even supports Shared Object rules [13], which is not part of this project due to complexity. However, both of these programs suffer from the fact that they are command-line oriented. They are both very good at what they do, but lack the usability and efficiency offered by GUIs with buttons. These programs are a good choice for rule management in a small environment with one or very few sensors, but once the environment grows larger, with tens or even hundreds of sensors, they become ineffective.

2.5 Research on Software Performance

Since this project also investigates software performance, it is relevant to take a look at what others in the field write on the subject. This subject is missing from most software development reports from Gjøvik University College, which can be explained by the usual high focus on agile[8] development. [14] mentions that agile programming is becoming mainstream and by its nature is a concern for the future of software performance engineering, as the primary focus of an agile project is on functionality.

A proper performance testing methodology was hence not immediately available for this project. However, the articles [15] and [16] gave a few pointers towards developing the test cases described in chapter 8. Both articles also stress the importance of performance testing during the development rather than after it, as this allows for time to actually implement optimizations and improvements. The performance testing in this report does not strictly follow any methodology, but is a result of the team members' use of common sense and the scientific method.

Early on in the project it became clear that Snowman is very I/O bound, as it does a lot of database operations. The tests in this report show a noticable difference in performance between traditional rotational hard drives and SSDs. Myers speaks of this in his Master thesis [17] which shows that SSD has a significant advantage over HDDs when it comes to random read performance but not when it comes to write performance. Note that these tests were performed in 2008, a time where SSD technology was fairly new and suffering from their different way of writing data[3]. As operating systems, drivers and disk controllers has improved, SSDs are now superior to HDDs in most tasks.

[2] In the five digit range.
[3] Features such as TRIM was introduced shortly after Myers thesis.

3 REQUIREMENTS SPECIFICATION

Many of the requirements for this system stems from functions that already reside in BHTB, which needs to be reproduced or improved. This means that the requirements for this project is closely tied to the way BHTB operates and its general workflow. This section will outline the specifications gathered from BHTB and from conversations with BKI.

3.1 Functional Requirements

3.1.1 Program Workflow

To aid the development of the functional requirements, the program workflow was charted in order to give a better understanding of what the system was expected to do. This was accomplished by going trough the functionality of BHTB step by step with the client and detailing the functions and various needs for improvement or additional functionality. From this, the workflow chart was drawn with the major parts of the system. In addition, Henriksens management process [3] was taken into consideration.

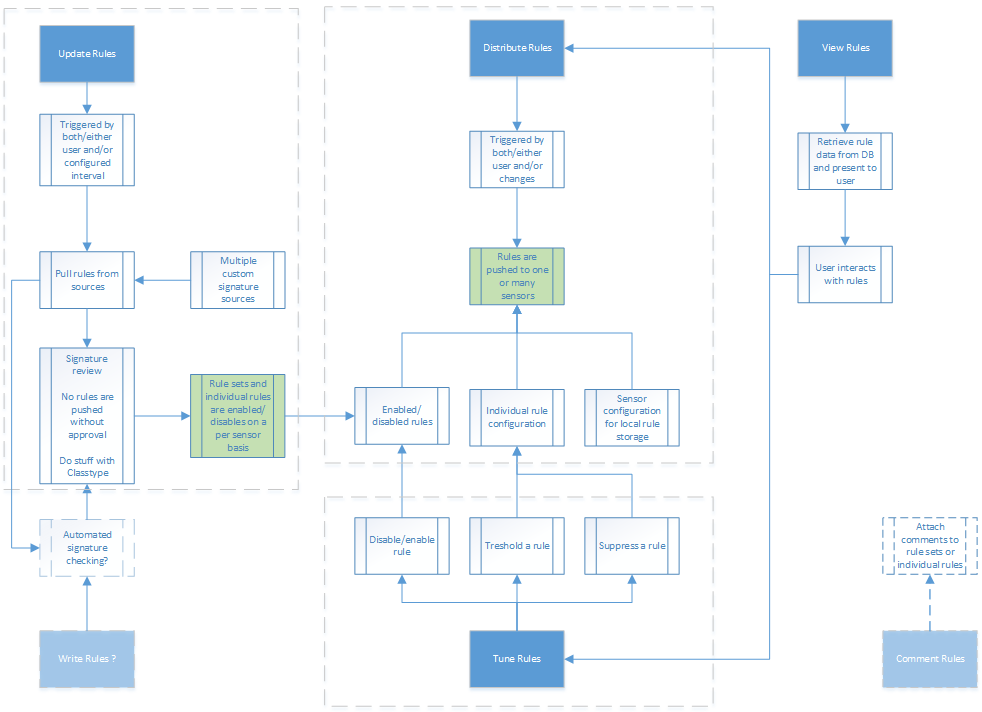


Figure 2: Program Workflow

To explain the workflow chart in figure 2, the blue boxes are the major system functionalities, the green boxes are the major goals of the system and the arrows show the direction of flow the system should have. As can be seen, all of the program flow ends up

with rules being syncronized to the sensors, which makes sense since this is the whole purpose of the system to begin with.

But the road to Rome consisted of a lot more functionality that needed to be in place, and that interacts with the rules, before they are ready to be handed over to Snort.

It's important to note that very early on, we decided that writing rules, automatically check them for errors and commenting them were nice-to-have (but still important) features that we could implement if we had time to do so. Hence they are greyed out in the drawings, and as you will notice further on in the report, was never developed due to time contraints.

From this workflow we could outline a rough modular design (figure 3) based on the naturally separate functions of updating rules, tuning them, distributing them and show them. We also added a database module, since most of our program was going to be centered around it.

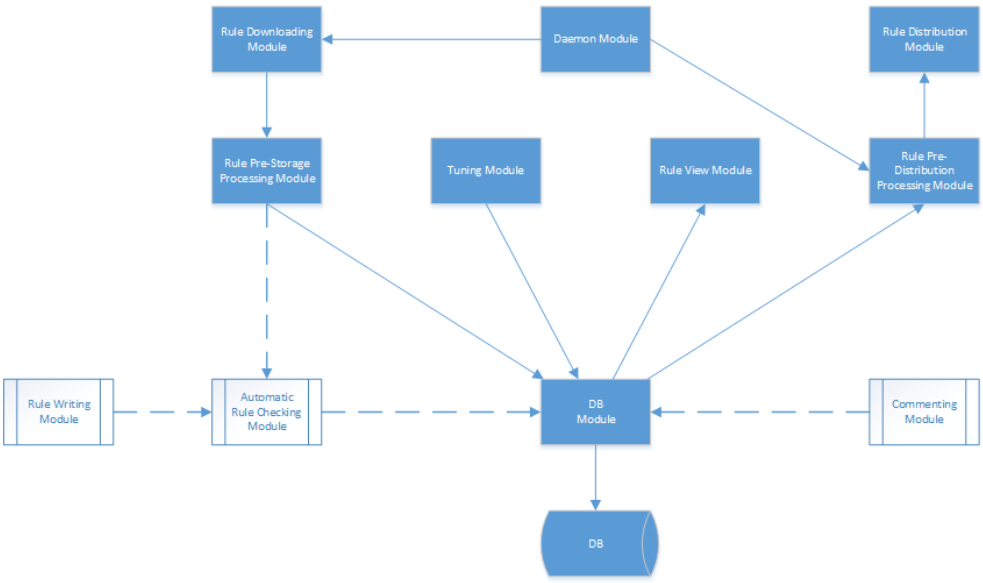


Figure 3: Program Modules

Rule Downloading Module

This module will handle all functionality related to downloading rules, including comparing MD5 checksums.

Rule Pre-Storage Processing Module

This module will essentially take downloaded rules, and parse and process them into objects that can be fed into the database.

DB Module

This module will handle all input and output to/from the database.

Rule Pre-Distribution Processing Module

This module will essentially take rules stored in the database and prepare them for distribution to sensors.

Rule Distribution Module

This module will be in charge of distributing rules out to the Snort sensors and handing the rules over to Snort itself.

Tuning Module

This module will handle all functionality related to turning rules on or off, filtering them or suppressing them.

Rule View Module

This module will contain all the functionality needed to display the system status and system functions to the user. It will be the primary way the user interacts with the system. Basically the GUI module.

Daemon Module

This module will run continually on the system to ensure that the system is responsive and to carry out any interval based tasks, such as automatic updates or distributions.

Commenting Module

This module will allow users to attach comments to rules.

Rule Writing Module

This module will handle functionality for writing custom rules and editing existing rules.

Automatic Rule Checking Module

This module will be responsible for verifying rules to ensure that there are no malformed or misconfigured rules that may potentially break or cause a halt on a Snort sensor.

3.1.2 Use Case Diagram

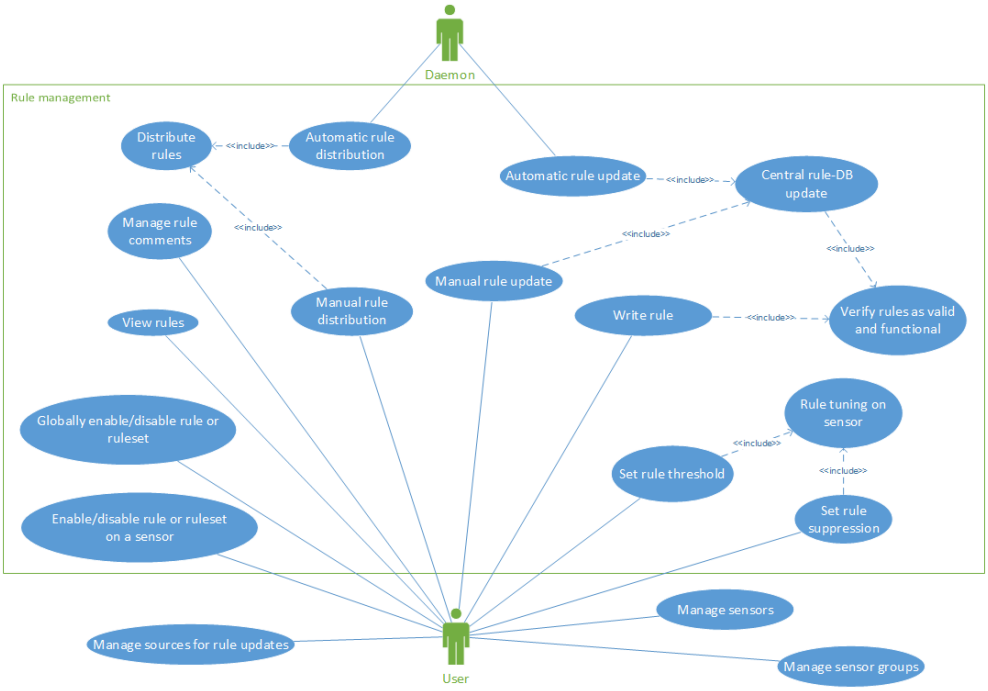


Figure 4: Use Case Diagram

3.1.3 Domain Model

After outlining all the things above, we put together the concept model in figure 5 of the objects we would need in the system. Even though a Snort rule is quite simple in and of itself, there are a lot of meta information surrounding them, like rule revisions and rule classes. Thus our system needed at least 16 different classes in order to fullfil the basics of the required functionality.

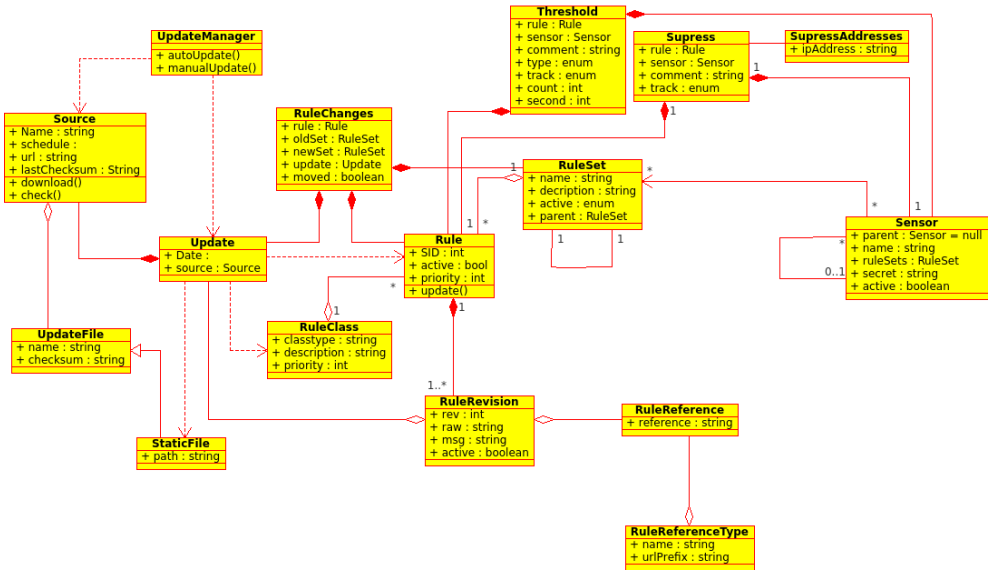


Figure 5: Conceptual Class Diagram

3.2 Supplemental Requirements

In addition to the features discovered in the use cases, we added had some requirements we felt were needed, that can be broken down into a RUP FURPS+[18] fashion.

3.2.1 Functionality

- The system should be able to restart a Snort sensor in the event that one crashes.
- The system should be able to detect if a Snort sensor is not responsive over the network.
- The system must be portable to the majority of the popular linux distributions.

3.2.2 Usability

- The graphical user interface must be simple and clean, but still contain all the features any user might require to interact with the back-end system.
- The system interfaces, herein API and function calls, must be self-explanatory.

3.2.3 Reliability

- The system must not cause a SNORT sensor to crash.
- The system must be responsive to API calls at all times. In the event the system capacity limit is reached, new requests should be queued.

3.2.4 Performance

- The system should handle administration of at least 200 sensors.
- The database should be able to maintain at least 100 000 individual rules.
- Any operation in the system in normal operations (i.e. after the first big rule update) should take no longer than 1 second on average.
- The relationship between time and workload should be linear, meaning that if a single operation that takes 1ms, ten operations should take 10ms.

3.2.5 Security

- The link between the system server and the remote sensor must be secured in such a way that a man in the middle cannot read the transmission in plaintext.
- The remote sensor and the system server must verify that they are trusted parties in the system, through authentication.
- Certain important API calls must be protected against misuse from a non-trusted party.
- All processes and files related to them system must exert the principle of least privilege.

3.2.6 Interoperability

The system must be able to work with Snort in such a way that it can import new files to Snorts configuration files structure and must be able to reset Snort in such a way that the new configurations are loaded.

3.2.7 Licensing

The system will be released to the public as Open Source under the GNU General Public License v3 license[19].

3.3 Constraints

3.3.1 Platform

As mentioned in the Project Plan, Snort is primarily used in conjunction with the Linux/UNIX operating systems. For that reason, the project was restricted to only develop for that platform. More specifically, we limited ourselves to the Ubuntu and Debian operating system, which is the primary platform the client uses as well. We have, however, stived to make the software portable.

3.3.2 Data security

While the system itself will not handle any data that may comprise of personal data, it may contain rules and rule sets that are considered classified or confidential. The system has therefore been built with this in mind and ensures confidentiality within the boundaries of the system. The system does not guarantee the confidentiality of the system outside these boundaries.

These boundaries are defined as the program and its processes and threads, and the network link established between the system and remote sensors. The system assumes that the users have already established database security.

3.3.3 Graphical User Interface

The project has had its main focus on the systems functionality and efficiency, and developing a graphical user interface (GUI) has been considered a secondary task and was prioritized as such. None the less, a GUI for demonstrational purposes has been included.

3.4 Security Assessment

During the inital phase we conducted a small security assessment in order to be aware of and possibly fix any security issues the software might have. We did a small brainstorming session and created a simple risk analysis table of the potential weeknesses we could find.

Threat	Component	Consequence	Probability	Risk	Strategy
Unauthorized users change data through the API/Web-interface	API/Web	8	5	6,5	Authentication/authoriz Django support for use and authentication.
An unauthorized user gains direct access to the database and is able to alter the data.	Central database	10	1	5,5	Assume database securi is in place.
Continually sending requests to the server, filling it up with requests to the point of causing a denial of service.	RPC Server	7	4	5,5	Authenticate requeste Rate-limiting on request logging.
An attacker inherits escalated privileges through exploiting the software if it is run as root or other high privilege user.	All processes	8	3	5,5	Follow least privileg principle. Python als protects against buff overflow
The SID/Rev of a malicious rule is newer than an excisting rule, with the purpose of overwriting the excisting rule.	Update module	7	2	4,5	Add possibility for th user to see changes befo they are commited.
In a scenario where the database is not on the localhost, a man-in-the-middle attack is possible.	Central database	6	3	4,5	Assume database securi is in place, encrypt tran missions.
Continually sending requests to the sensor client to update itself, filling it up with requests to the point of causing a denial of service.	RPC Client	5	4	4,5	Limit the allowed upda frequency, logging.
Some unauthorized machine poses as a legitimate sensor and gains access to rules through update requests.	RPC Server	5	2	3,5	Authenticate the sensors
During an update, one or more rules contains errors that causes Snort running on a sensor to crash or shut down.	Update module	8	4	6	Verify the rules.
A rule file is specially crafted to contain an SQL injection.	Update module	10	2	6	Use prepared statemen and never let input d rectly touch the databas
SQL injections, XSS og CSRF (cross site request forgery)	API/Web	5	5	5	Django has support fo protection against thes attacks. Validate input.

Table 2: Security Assessment

4 CONCEPTS OF SNORT

This chapter will give a brief introduction to the basic concepts of Snort.

4.1 Rules

A rule is basically a set of options that define ways of detecting, classifying and reporting a potential attack or threat on the network[1]. There are several types of options, but only the general rule options are relevant for this project. The general rule options provide information about the rule [13] (i.e. revision number, alert message). Other types of options, such as payload options which are actively used in detection, are not relevant for the management of rules and are thus not considered in this project.

A rule can either be enabled or disabled in the detection system, meaning that the rule will only have an effect on detection when it is enabled. Since this project is dealing with multiple sensors connected to one central database, this functionality has been expanded so that users can keep a rule enabled on one sensor and disabled on another. In addition, the user can also set an enabled/disabled flag on *rule sets*.

4.2 Rule sets

The term rule set, sometimes ruleset, is not formally defined in the Snort Users Manual, however it is used a few times to refer to the complete collection of official Snort rules[2] and to the complete set of rules applied on a Snort system. In practice, rules within a rule set are categorized by intrusion type (e.g. malware, trojan and worm). These groups of rules are often named by their intrusion type, and are also referred to as rule sets. Snowman and this report use only the latter definition of rule sets.

4.3 Generators

Generator is a term used for any subsystem of Snortwhich generates alerts from rules [13]. Each generator is identified by a unique generator ID. The rules subsystem, which has generator id 1, is the only relevant generator for this project.

4.4 References

Snort contains a plugin which allows rules to contain references to external attack identification systems which provide additional information about the alerts [13]. A reference is basically a URL string pointing to a webpage containing this information.

4.5 Event and detection filters

Filtering can be applied to rules in order to control the amount and frequency of alerts. Two types of filters can be applied to a rule: event filters and detection filters. A detection filter can only apear inline, whereas an event filter can only appear as a standalone

[1] A rule actually consists of a header and options [13]

[2] Also referred to as the Sourcefire VRT Certified Rules, see http://www.snort.org/snort-rules/

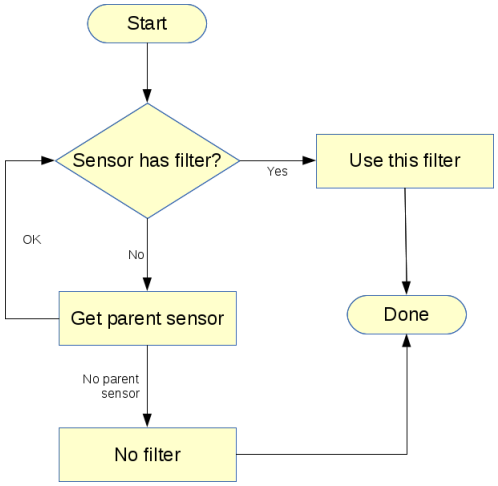


Figure 6: Flow

option[3].

All filters operate with a *count* and *seconds* variable, representing the number of events in a time interval and the duration of the time interval, respectively. Event filters are further divided into three types (table 3).

[3]An event filter can currently be written inline (called a rule threshold), but according to the Snort Users Manual this feature is deprecated and will not be supported in future releases.

Filter type	Alerts
Event filter	
Limit	One for every event when number of events $\leq$ C in interval S
Threshold	One for every C events in interval S
Both	One per interval S when number of events $=$ C
Detection filter	One for every event when *count* $>$ C in interval S

Note: C and S represent the count *and* seconds *values in the filter.*

Table 3: Effects of different filter types.

5 DESIGN

This chapter will present the design specification of Snowman.

5.1 General Design

This section is deicated to the discussion of the general design of Snowman.

5.1.1 Centralized or distributed setup?

We decided early to create Snowman in such a way that it can be set up in a distributed manner. The main considerations behind this choise is the bandwidth-requirement needed by the rule-updates, and the load of the central server.

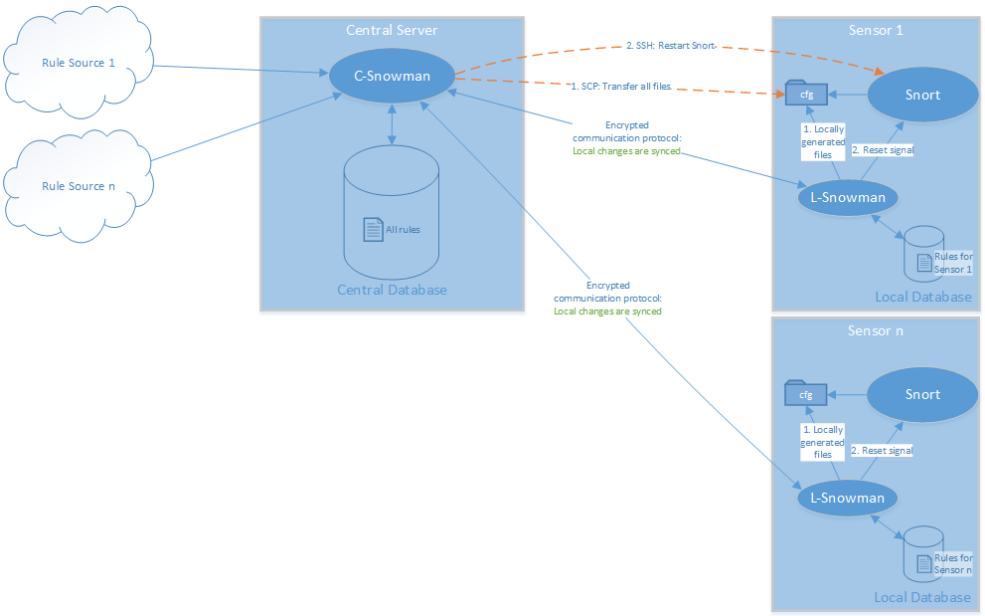


Figure 7: General architecture

On figure 7 you can see the difference between central and distributed setup. The red dotted lines represent the communication in a centralized setup, while the blue lines between the server and the clients represent the communication in a distributed setup.

Bandwidth requirements

The Snort sensors managed by Snowman might have very varying network-infrastructure connecting them to the central server. In worst case, the sensor might be placed at a military-base in the field, on the other side of the planet, only connected to the central server via a sattelite-connection with limited capabilities for transmitting data. As the rulesets applied to sensors typically is tens of thousands rules, and the changes between two versions of the sets typically affects about a thousand of them, it seems reasonable to only copy the changes when an update should be done.

Central server load

When a full update of all the sensors is requested, new rule-sets would be needed to be created, and distributed to the sensors.

In a centralized enviorment, the Snowman server would need to generate all the new rule-sets, and send them to the client. The server would not known on beforehand which rules currently exists on the sensor, so a full list of rules would need to be sent. Even if not a single rule changed. Also, if the sensor for some reason is unavailable, or connected trough a unreliable network-connection, the central server would need to put down quite some effort to send the rules to the sensor. Again, even if it is not neccesary as no rules have changed.

In a distributed eniorment, we would be able to put some logic (a client) onto the machines hosting the sensors. This way, when an update is occouring, the client can get a list of rule-ID's from the central server, and self be responsible for collecting the new revision of the rules that have changed, or is missing. This way, the central server would only need to collect the full rules from the database if the rule actually is needed by the sensor.

5.2 Server design

The server is the central component of Snowman and generally consists of:

- Central rule storage
- Web-interface for user interaction
- Update-module to get updates from rule-sources
- RPC-interface to communicate with the clients

The central storage of rules would be in a normal relation database. As we have decided to use Django as our framework, we have a great flexibility to which database we actually use. In the current version of Django (1.6) we might use MySQL, PostgreSQL, SQLite or Oracle.

For the RPC-Interface, we have decided to use XML-RPC. This is a very simple protocol using XML over HTTP to transfer requests and data to and from the server. To be able to keep an aspect of privacy, we encrypt the raw HTTP stream using SSL.

5.2.1 Architecture

We have designed the server by the layered approach of MVC (figure 8).

The web-interface uses pure Django logic. The HTML visible in the users browser is generated by the Django template-engine, which recieves its data from the views created by using ordinary pyhton code.

The XML-RPC server uses a handler which is administering the connections from the sensors, and exposing the relevant methods to allow the sensors do a synchronization.

The update is unpacking and preparing the update-files for parsing. It then handles the files to the parser. The parser is then handling the different parametres to the Updater, which in turn saves them to the database in an efficient manner.

Finally, common for all the interfaces is the Django Models. We use the database-abstractionlayer from Django for mapping the lines of a relational-database to django

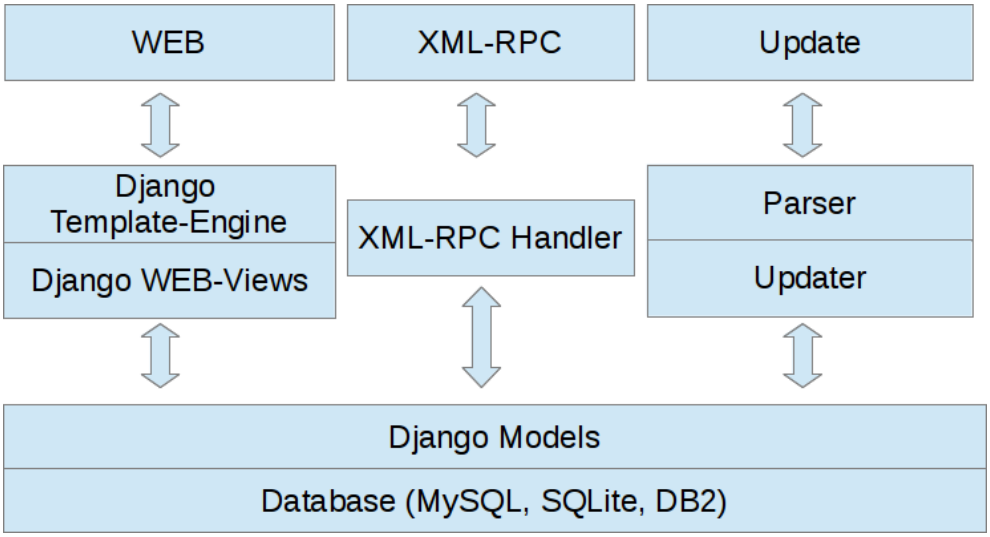


Figure 8: Server Architecture

objects. This is the part that lets us write code once, and still be able to use several different database-engines.

5.2.2 User interface

The main user-interface for snowman is the web-interface. It is based on django's template-system for the generation of html which is displayed in the web-browser on the user's computer.

The web-interface uses javascript to dynamicly be able to load/change the content of the pages without refreshing the whole page. Important parts of the webinterface is possible to download using AJAX, to update tables and simmilar.

Javascript is also used to improve the user-experience of the interface, due to functionality which a moden computer-user expects from modern web-pages.

Url's

The first step of serving a page to a user is a HTTP request for a specific page is recieved. The pages are uniqely identified by url's, like '/web/rules/'.

The url's are structured in a logical manner, where everything in the webinterface is found under /web/, everything regarding the administration of rules is found under /web/rules/, etc. The url's might contain dynamic data, like pagenumbers (ie: /web/rules/rules/page/3/ which will give you the third page of the rule-list), so that dynamic content can be loaded dynamicly when needed.

Django uses the urls to identify which views the requested url corresponds to. When a view is identified, the request (and its dynamic content) is passed to the corresponding view.

Views

The views is the part of the user-interface which collects the required data from the models-layer, and generates a response to the web client. The response can contain a full web-page, or just a small subset of one if the request was issued by AJAX.

The typical steps a view makes, to generate a response would be:

1. Collect the requiered data from the database(s) using the Django models.
2. Hand the collected data, and a template, to the template-engine to generate the HTML representing the webpage the user want.
3. Send the created webpage as a response to the user.

The view-system decides what to show, and then lets the template-system decide how to show it. The view-system in itself is not creating any html or simmilar. It soley creates data-structures.

If the request was for some data, and not a webpage, step 2 in the list above might be skipped, and instead return the data as JSON or simmilar in step 3.

If errors oocures in the view-system, appropriate exception can be raised to tell Django to return an error-code or an error-page to the user, instead of what the user requested.

Templates

When the view-system have prepared all the data, the template-system takes over. The job of the templates is to take the supplied data, and inject it into pre-made skeletons of HTML. This way, all the design/display part of the webinterface is collected to one single place, and that give us a very clear line between data and design.

5.2.3 Database

As the database of the server is quite complex, we have divided the drawings, so that we have one drawing per module.

Core

The core database-architecture used for storing the rules is described in figure 9.

The most central part of our system is the representation of a Rule. A single Rule might have several revisions, as we might want to keep some history when they get updated. Each revision have its own set of references attached.

A single rule also have a lot of other properties which it might share with other rules. A generator, a classification and a ruleset are examples of such.

Finally, the rules are placed onto a sensor by attaching ruleSets to the sensors.

Tuning

In addition to storing the raw rules, we would like to be able to store tuning-parametres on the rules. As sensors might have different tuning-parametres, we cannot add the tuning directly to the rules. Own tables (as described in figure 10) is hence needed to model the tuning-parametres.

Both types of filters, and the suppresses is connected to a Rule and a Sensor. This way we let the user spesify filters for the individual sensors if that is desired. If a sensor does not have a filter applied, but any parent of the sensor have one, the closest parent's filter will be applied.

Update

When rules are inserted to the database, we would like to have some traceability to be able to determine from where a rule have entered our system. To be able to do this, we have the update-module (figure 11)

The structure of the database to update would let us store the update-sources, and add a update-object for each update ran from a given source. These update objects can

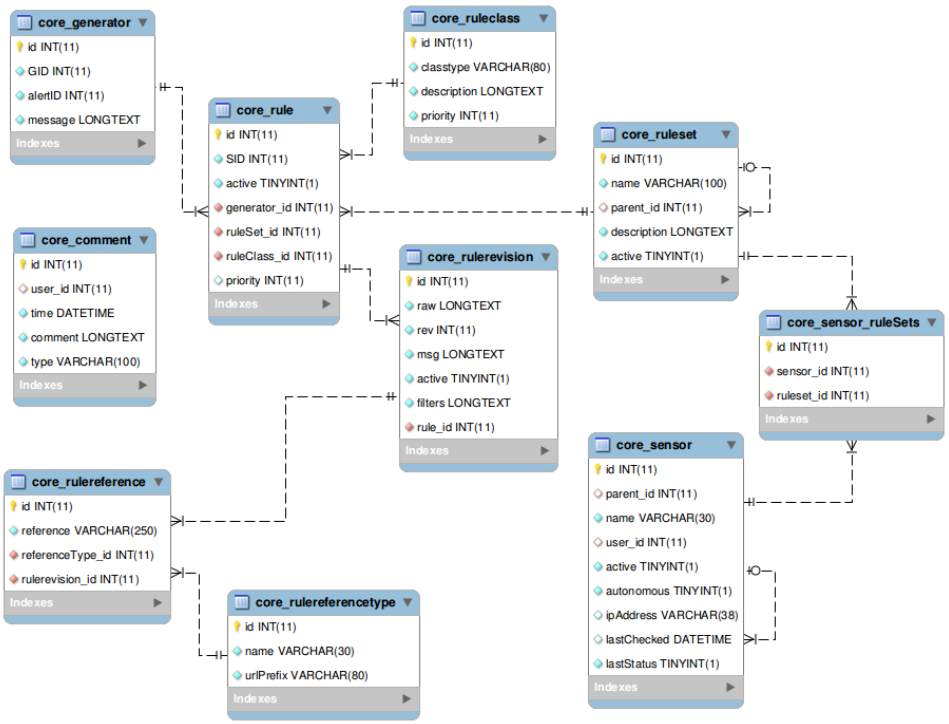


Figure 9: Core server-database

in turn have a relation to rules, rulesets and rulerevisions, so that we can track what changes for each update.

5.3 Client

As the client is just going to take care of getting rules for a single sensor from the central server, and should just apply these to one sensor, the architecture of the client can be much simpler than the one of the server.

5.3.1 Architecture

The MVC schema for the client is to be seen in figure 12.

The update-handler of the client have both a XML-RPC Client and a XML-RPC Server. The client is used to connect to the central server, to do a rule-synchronization. The rpc-server is here so that the central server can connect to it and check that the client is alive, or to order an update of the rule-sets.

As the client should be as light-weight as possible, and the only thing we would like to have a framework for is the database-connection, we have decided to not pull inn Django for the client. Instead we use the database ORM for python called "SQL-Alchemy".

5.3.2 Database

The database on the client would be significantly simpler than the one on the server, as we only care about the rules which should be present on a given sensor. The full schema is available in figure 13

The central part of the datastructure is the rule table. This table contains all the parametres directly related to the rule. In addition we have seperate tables to all the

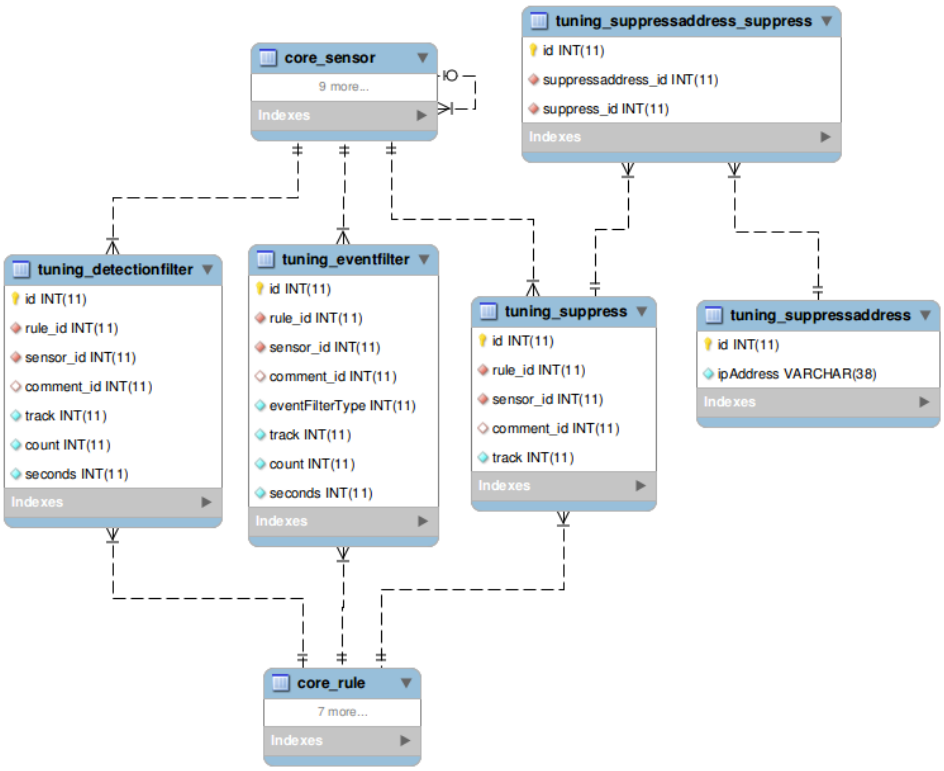


Figure 10: Tuning server-database

parametres modifying the rule (like filters), or that is common between several rules (like rulesets).

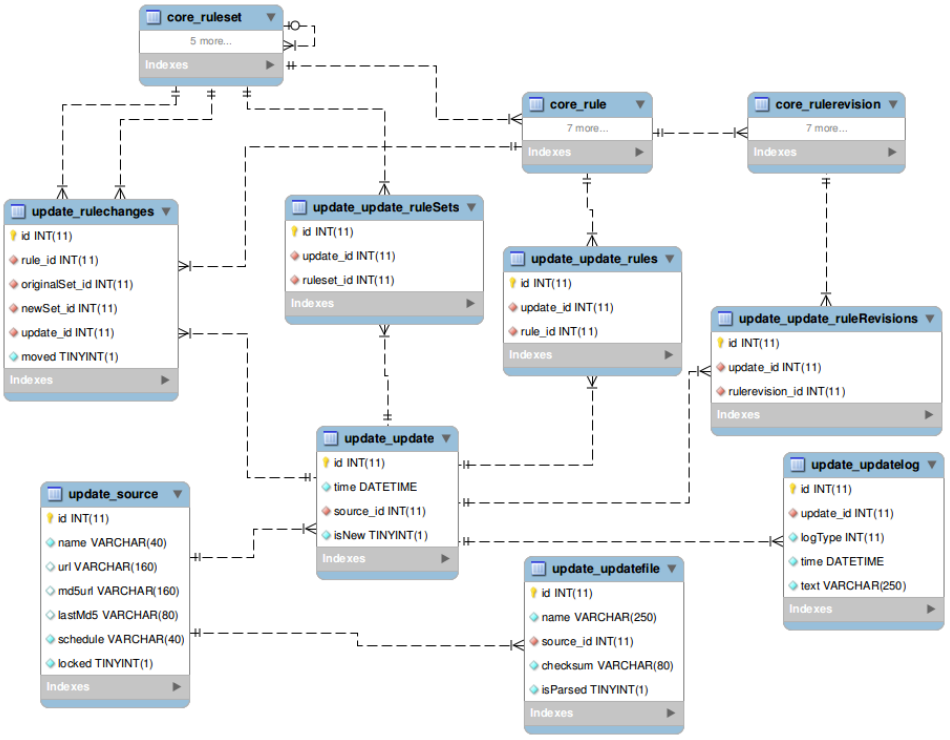


Figure 11: Update server-database

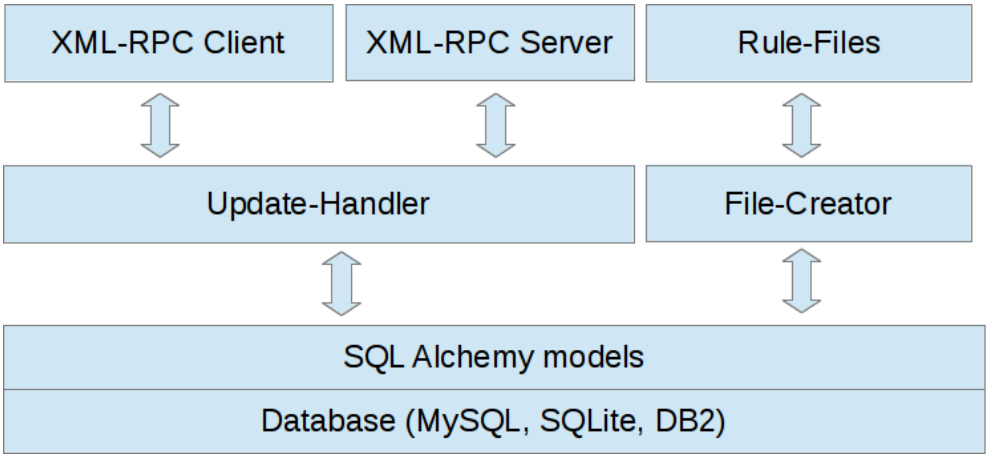


Figure 12: Client Architecture

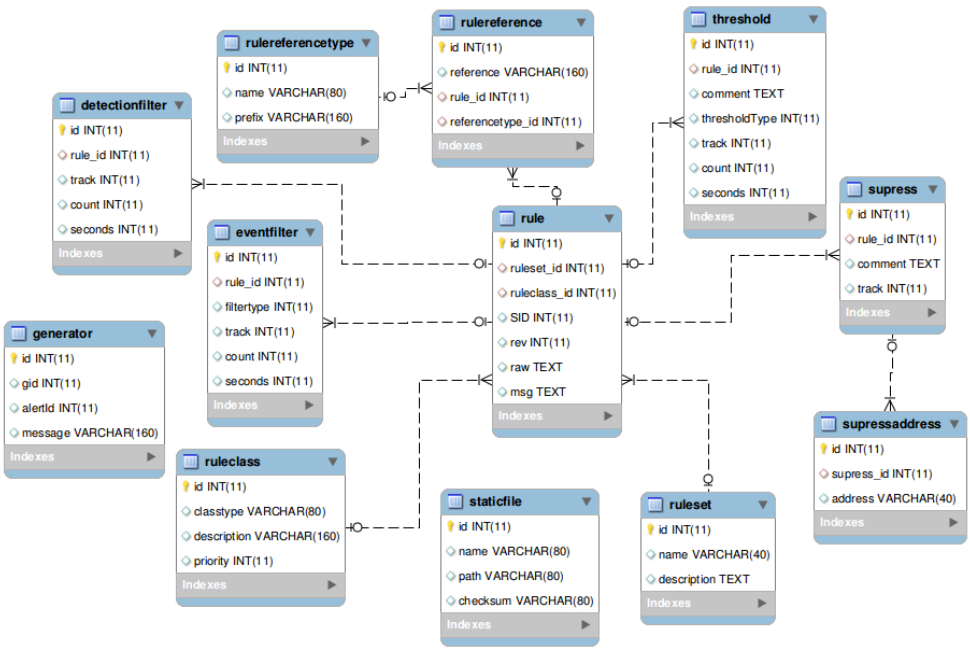


Figure 13: Client Database

6 IMPLEMENTATION

This chapter will focus on the technical implementation of Snowman. The first sections cover programming languages, development tools and frameworks used in this project. the most important modules including the GUI. Application logging and the documentation and deployment of the software is covered at the end of the chapter. Some sections within this chapter require a general knowledge of the key concepts of Snort described in chapter 4.

6.1 Programming languages

Snowman is primarily written in Python 2.7 [20]. The project group landed on this decision after discussing the aptness of C, C++, Java and Python to this project, with regard to development and processing efficiency. Due to the limited time-frame of this project, it was vital to get right into the development without having to learn a new programming language or unfamiliar programming concepts. It was also important to find a language which produces high performing programs, since this project aims to be more effective than similar software. The C and C++ languages were considered because these languages can produce programs with very high performance. The project group also has some theoretical and practical experience with these languages, as they are taught at Gjøvik University College. C and C++ were abandoned as the team concluded that they would induce more work because of their complexity, and thereby be too time-consuming.

The final decision fell on Python mainly because of this language lets the programmer develop complex programs in a short amount of time. Python has automatic memory management and offers a wide range of built-in tools, which gives the developers time to focus more on the code and functionality rather than overhead tasks like managing memory and including external libraries.A concern was raised that Python, being a high-level language, would maybe not be efficient enough for this project. This turned out to not be a problem, as python can run modules written in C. A major reason for using Python was also Django (see section 6.3), which is a Python framework used in this project that greatly simplifies database transactions and generation of web pages.

The most current version of Python available at the start of this project was version 3.3. The project group selected version 2.7, as a break in compatibility [21] between 2.X and 3.X versions of Python cause some problems. According to a recent survey [22] Python 2.X still holds a dominant position among developers, five years after the release of Python 3.0. According to [21], version 3.3 has slightly worse library support. The continuing popularity of 2.X versions implies that many packages are still being developed and maintained for these versions, and these versions only. One example of this is the aforementioned Django framework which is heavily used by Snowman. Development using Python 3.X with Django is possible with the use of a compatibility layer [23], but the project group considered this as a bit cumbersome. Taking into account that the majority of the group was already comfortable with Python 2.X, this came out as the most

time-saving choice. The biggest drawback of this decision is that it does not make Snowman future-proof, and it will probably have to be ported to Python 3.X in a long-term perspective.

For Web-programming, this project has used HTML and CSS for the design and JavaScript for client-side scripting.

6.1.1 Standards and guidelines

Python imposes a strict indentation regime, as this is actively used in the syntax to delimit blocks of code. The project group has followed standard Python guidelines for best practice programming, with principles as DRY[1] and EAFP[2] (the latter actually caused some unforeseen performance issued described in chapter/section).

Documentation has been a high priority. The project group has aimed to make the code self-documenting by utilizing sensible names on variables, functions and classes to enhance the reader's understanding. As a general rule, variable names are written in camelcase[3]. With a high focus on development, the project group has taken advantage of Python Docstrings [24] (documentation strings) which are incorporated into the code, and can therefore be written at the time of coding without taking the focus away from the programming. The Docstrings describe how the code is used, and form the basis of Snowman's documentation. Listing 6.1 demonstrates the EAFP-principle, also note the use of a Docstring at the beginning of the method.

The Python manual encourages the use of exceptions, which is a useful way of sending messages up the execution stack when an error or irregular behaviour is deceted. Customized exeptions are raised in Snowman for example when parsed data has the wrong format (*BadFormatError*), when critical objects are missing from the database (*MissingObjectError*) or when an unsupported rule is encountered (*AbnormalRuleError*).

Writing object-oriented code has not been a goal in this project, but concepts such as encapsulation, polymorphism and inheritance have been applied when needed.

```
1  def getSecondMessage(messages):
2    """Returns the second message from messages or
3    empty string if second message does not exist."""
4
5    # Below code demonstrates the principle of
6    # EAFP (1) versus LBYL (2); The execution
7    # of block 1 is faster than block 2 when
8    # message[1] exists. Block 1 is therefore
9    # preferred whenever the try-block is
10   # expected to succeed without an exception.
11
12   # Block 1, EAFP:
13   try:
14     message = messages[1]
15   except IndexError:
16     message = ''
17
18   # Block 2, LBYL (look before you leap):
```

[1] "Don't repeat yourself". The principle that code should not be duplicated in a project.

[2] "Easier to ask for forgiveness than permission".

[3] A style convention where multiple words are written as one, with the first letter of each word capitalized, e.g. getAllRules.

```
19    if len(messages) >= 2:
20      message = messages[1]
21    else:
22      message = ''
23
24    return message
```

Listing 6.1: The EAFP-principle.

6.2 Development environments

This section describes the various development environments and tools used in this project.

6.2.1 Environments

Snowman is developed on and for the Linux platform. (can it be easily ported?). Eclipse was selected as the default development environment, as this is freely available and contains the major features required for effective development such as project management, a Python editor and a debugger (available with the PyDev add-on). PyUnit has also been used for unit testing. In addition, the TeXlipse and PDF4Eclipse add-ons for Eclipse have been used to write and generate this report with Latex. Since Python is an interpreted language, the Python interpreter has been used throughout the project to test and debug code segments. (Thomas: Web-browsers, web-debugging etc.) (Other tools, software used?).

6.2.2 Version control

This project has used Subversion (SVN) provided by the IT department at Gjøvik University College for keeping version control for all project files. This includes all source files and project documents such as design documents, meeting minutes and the report. The SVN repository hosted by the IT department has also been used to ensure that data is not lost.

6.3 Frameworks

6.3.1 Django

To facilitate database transactions and the creation of the Web interface, this project uses the Django[5] framework. Django is a Web framework which takes care of database modelling and transactions, and user authentication. (Eigil?)

6.3.2 SQLAlchemy

Used on the client side database abstraction... (Eigil?)

6.3.3 jQuery

When it comes to using Javascript for client side scripting, jQuery[25] is the way to go. jQuery provides a framework for common javascript operations on the DOM and abstracts away a lot of browser spesific code to give you a nice and clean interface. It also makes AJAX class a lot easier, requireing only code such as

```
1 \$.get('/web/rules/');
```

instead of

```
function loadXMLDoc() {
    var xmlhttp;

    if (window.XMLHttpRequest) {
        // code for IE7+, Firefox, Chrome, Opera, Safari
        xmlhttp = new XMLHttpRequest();
    } else {
        // code for IE6, IE5
        xmlhttp = new ActiveXObject("Microsoft.XMLHTTP");
    }

    xmlhttp.onreadystatechange = function() {
        if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
            document.getElementById("myDiv").innerHTML = xmlhttp
                .responseText;
        }
    }

    xmlhttp.open("GET", "/web/rules/", true);
    xmlhttp.send();
}
```

6.3.4 Bootstrap

Since no one in the group can call themselves a web designer, we decided to take a shortcut on that regard and utilize Bootstrap[26], which is a front-end framework derived from Twitter. It provides us with a lot of ready made CSS and design elements that makes a web page responsive and typographically readable.

Consider the example job of creating a <table> element and spending hours setting up and tweaking the CSS for the said table element, we could just add <table class='**table table-bordered**'> and get the table formatted nicely with colors and fonts that are typographically matching.

6.4 Core module

This section will focus on the classes and methods in the core module. This module contains the most vital classes of the Snowman server including **Rule** and **RuleRevision** which hold the data of each rule in the system. Simple classes in this module such as **Generator**, **RuleReference** and **RuleClass**, which are connected to the **Rule** class are only briefly explained in this section. For an overview of all objects and functions in this module, refer to class diagram (ref) or appendix (ref).

6.4.1 Rules and rule revisions

Rules in Snowman are split into two parts; the rule and the rule revision. As threats change, or errors in the rules are discovered, a rule may become obsolete and a new revision of the rule is released. The rulestring, message and detection filter might change from one revision to the other, and are stored in the rule revision object along with the revision number. A revision is always connected to a rule object which contains data that is constant for all its revisions, such as signature ID and rule set. The reason behind keeping rules and revisions in two separate objects is to avoid writing the same constant rule data for all revisions. Users of Snowman have the possibility to control how many

revisions to store per rule, and whether incoming revisions should be active or inactive by default. This feature allows users to control automatically downloaded rules before they are put into the detection system. Users can also manually activate or deactivate rule revisions, and Snowman will distribute only the newest active revision of a rule.

6.4.2 Rule sets

As stated in chapter 4, the categorization of rules in rule sets is a de facto way of organizing rules. Rules in the community rule set[4] have a "ruleset" option within the "metadata" option in the rulestring which specifies the rule set. In other releases, rules are organized into files where the file name denote their rule set. These two ways of specifying the rule set are both supported by Snowman.

6.4.3 Sensors

Every sensor in the system is represented in the database with its name, IP-address and status. Sensors also contain a list of rule sets applied to them, and active rules within these rule sets will be distributed to the sensors given that the rule sets are also active. A sensor can be configured to be autonomous, meaning it will not communicate with the server. Autonomous sensors can be used in environments where contact between the server and sensor is unwanted or not possible. Rules can be manually exported for these sensors.A sensor can also be the parent of other sensors, allowing users to create groups of sensors with the same configuration. The database also contains an "All Sensors" object which is the parent of all sensors.

(Eigil: active, checked, status, secret?).

6.4.4 Comments

The `core` module contains a comment object which can store comments from users who wish to document their operations, e.g. the user can attach a comment when adding a filter to a rule to describe why the filter was added and which effects it will have on the system. Snowman also generate comments for important events, such as (Thomas?).

6.5 Update module

The purpose of the `update` module is to fetch and read rules and configuration files from external sources, and populate the database with this information. Effectiveness in updates is a key issue, and a number of deliberate optimizations for performing updates, with the goal of saving time by omitting unnecessary operations, are explained in the subsequent sections. To avoid confusion with the module, the term *update-package* is specifically used to refer to update data coming from external sources.

6.5.1 Sources

A *source* is a supplier of update-packages. Snowman lets users define sources which are either external (i.e. online) or local, meaning that the update-package will either be downloaded from the Web or read from the local system. All information about each source is contained in the `Source` class. The sources determine where the update-packages are located, and an update-package must always come from a specific source.

External sources contain a URL pointing to the location of the update-package, and alternatively also a URL to the MD5-hash of the package. External sources can be config-

[4] "The Community Ruleset is a GPLv2 VRT certified ruleset that is distributed free of charge without any VRT License restrictions" - http://www.snort.org/snort-rules/

Data field	Description	Presence
sid	Unique identifier for a rule.	required
rev	Rule revision number.	required
msg	Notification text when rule triggers.	required
classtype	Connects the rule with a specific class of attacks.	required
gid	Generator ID to identify which subsystem generates the event where this rule fires.	optional
ruleset	The ruleset this rule belongs to.	optional
priority	Override default severity set by classtype.	optional
reference	Reference to external resource with information about this attack.	optional

Note: The *presence* column indicates whether a field is required or not by the parser in this system. Information is gathered from the Snort Users Manual [13](ch 3.4).

Table 4: Rule options used in this system.

ured to pull updates at regular intervals, contrary to local sources which must always be updated manually.

A source can be configured to update at regular intervals, or the user can run an update manually on a source. To make the update process more effective, `update` makes use of an MD5 URL which points to a file containing the MD5 hash of the update-package. This hash is stored for every source (if available) and is compared to any pre-existing hash to verify if the system is up to date or not. `update` will only download and process an update-package when it encounters a new hash.

6.5.2 Files

An update-package may contain various files depending on the source it comes from. This section will describe the files supported by Snowman.

Rule files

A rule file is any file carrying the *.rules* file extension. These files contain raw rulestrings to be processed by Snort. A rulestring is basically a semicolon-separated list of rule options. Specific rule options used in this system are listed in table 4. A badly formatted rulestring given to Snort will cause a crash, and Snort must be restarted.

Classification, Generator and Reference Configuration files

The classification files are identified by their name *classification.config* and contain rule classification strings that are used by Snort to categorize event data. A classification file will normally appear once per update-package, and rarely changes between updates[5].

Sid message files

6.5.3 Running an update

An update is either triggered by the user, or automatically on regular intervals, for a given source. Regardless of how the update process is invoked, it follows the same procedure.

[5]https://www.snort.org/vrt/snort-conf-configurations/

As mentioned above, sources will often provide an MD5-hash of the update. When this is the case, the hash is stored in the database. Every new update will compare its own hash to hashes stored in the database, and if an identical hash is found this means that the update has been run before and the update process is terminated. When the hash does not pre-exist in the database, or for sources which does not use hashing, the update-package is downloaded to a temporary directory.

Snowman accepts update-packages in the form of tar-archives[6] containing rule and configuration files, and also supports plain text files containing rules or known configuration entities[7]. Tar-archives are extracted, and folders are traversed recursively

each file in each folder is parsed recursively. For added efficinecy, Snowman creates a hash of all files which is stored in the database. This is later used to see whether any files in a new update has not changed.

In most cases the update-package will contain only minor changes in relation to its predecessor, e.g. only one file is different in the latest package, and parsing all files that has not changed would therefore be a waste. To address this issue and save processing time, the name, path and MD5-hash of every file is stored in the database per source. Every time the update module parses a file, it first creates a hash of the file and compares this with the respective file's hash in the database. If the file is present in the database and the two hashes are identical, the file is considered *not new* and is skipped. The following sections will elaborate on the parsing of files based on file type.

skipping files, sequence of files.

parse files with regex.

6.5.4 Rule parsing

Some options can be defined both in the rulestring (inline) and in a separate file (standalone).

If any properties classified as *required* for a rule (table 4) are missing...

Non-existing rule sets, rule classes and generators are stored in the database

After the initial parsing of the rulestring, the parser checks if the rule already exists in the database, and if so whether the incoming revision number is higher than the latest revision of the pre-existing rule. The parsing will continue only in the event of a new rule or a newer revision of an existing rule.

rule moved to new ruleset.

Rule sets

A rule always belogs to a ruleset which can be specified in the raw rulestring. If the parser does not find this attribute in the rulestring, the filename is considered to be the ruleset name.

Threshold

A rule threshold defines a limit and/or threshold for the triggering of a specific rule, and can exist as an inline or standalone option. Rule thresholds are deprecated, and support for this functionality will cease in a future version of Snort, where it is replaced by *detection filters* and *event filters* (see below) [13]. To maintain compatibility with current rules, Snowman supports reading inline and standalone threshold definitions.

[6] Files that contain compressed files.

[7] Generators, filters, references etc.

Detection filters

Please explain..

Event filters

Please explain..

Suppression

Always standalone.

6.5.5 Classification, Generator and Reference Type parsing

These files contain a sequence of their respective entities (section 6.5.2), and are all parsed in the same manner. If a given entity is not already present in the database, it is created and stored. If the database already contains the entity, the existing entry in the database is overwritten.

Rule Message files

6.5.6 Saving

During early versions of the update-code it was discovered that the saving of changes to the central database would need some special care. The first versions of Snowman saved every rule individually, after a change was found, and the objects was updated. This was discovered to be a performance-problems, as updates could take hours on weak hardware.

A new approach was therfore planned, and now we store everything into memory while parsing, and only starts to save changes into the database when everything is parsed. As some of the objects is reffering to other objects in the database through foreign-keys it is vital to store the objects depending on other objects after the objects they depend on is stored. A safe order to save them is identified, and the objects are saved in this order:

1. Generators
2. Classifications
3. Reference-types
4. RuleSets
5. Rules
6. RuleReferences
7. Suppresses
8. Filters

For every point in the list above, the procedure is more or less the same. First, the objects recieved is attempted to be extracted from the database in one request. The response from the database lets us identify which objects are new, and which is changed. The changed objects can be updated and saved individually, before all the new objects are constructed, and inserted into the database in bulk.

6.5.7 Logging

in Snowman this is translated to one filter of each type per rule per sensor.

If a filter occurs inline, this filter is regarded as a default filter and will be active on every sensor where the rule operates. A user can choose to override the default filter on a sensor by defining a custom filter to be used on the respective sensor.

To stop Snortfrom adding duplicate filters, all inline filters are removed from the rulestrings. Any filters extracted from the rulestring are written as plain text (i.e. as they appear in the original rulestring) in the *filters* field of the rule revision object. This is to ensure that all the original rule options are stored meaning that no data is lost or discarded.

Filters are removed from the rulestring and transformed into event filter and detection filter objects.

6.6 Distribute

To distribute the rules from the central database to the sensors in a format that the snort-software can understand, three major things happen. Each of these is covered in the following three subsections:

6.6.1 Initiating the distribute

Before a synhronization can start, the sensors would need to know that they should synchronize. There are mainly two things that can cause a synchronize to start:

1. Automatic synchronization based on time
2. Manually invoked trough the webinterface on the Snowman-server.

In the first case, the client itself knows that it should start to synchronize, and therefor just starts the authentication/synchronization procedure with the server.

In the latter case, the client does not initially know that anything happened as the central server is what hosts the webinterface. The central server then contacts the client via the “snowmanclientd” daemon, telling it to initiate a synchronization.

6.6.2 Synchronization

When a synchronization is initiated, the client starts with connecting to the “snowmand” daemon on the sentral server, and authenticate with it. If the authentication is successfull, the server knows the sensor, and which rules the sensor should get installed, and we can then start the synchronization.

The first thing that is done is to retrieve all the RuleClass objects, and update the local cache to correspond to the retrieved list. Changed objects will be updated, new objects inserted into the local database, and obsolete objects will be deleted. Next lists of Generators, RuleReferenceTypes and RuleSets is retrieved, and the local database is updated in a simmilar manner as with the RuleClass objects.

For the synchronization of the rule themselves, which is the main bulk of the data needed to be synchronized, a more efficient approach is done. First a list over the signature-id’s and revisions of the rules that should be present on the sensor is retrieved. If rules in the local cache is not mentioned in this list, or this list indicates that a new version of this rule is available, the rule is deleted from the local cache. Eventual tuning-parametres belonging to this rule is also removed.

Next, the client starts to request the rules that it currently is missing from the server. To speed things up, this happens in bulks of 250 rules per request. This way, no requests gets too big and time-consuming, while we do not use too many requests of lots of things have changed. For the rules requested, tuning-parametres like filters and suppress is supplied if the rule have any such applied on the given sensor. References is also passed along with the rule.

All the recieved rules is then saved to the local database, and the synchronization is complete. The final step of generating snort configuration-files is then started.

6.6.3 Generating snort configuration

The generation of the snort configuration-files is a rather simple process. First, the old configuration is cleared out, and new files are generated based on the content of the local database.

The naming-convention of the files is simmilar to the one SourceFire supplies their rulesets, and the format of the files are exactly the same. This is done to simplify the visual inspection of the rulefiles, if that is desirable. Snort in it self does not really care of the configuration-files to be in a certain order, as long as the content can be recognized as snort-configuration.

In addition to the various files created by snowman, a certain file is guaranteed to be generated. That is the "snowman-includes.cfg" file, which names all the files generated. If snort includes that file, all the files generated by snowman will be included by the snort software.

Finally, when the configuration is generated, the snowman-client sends a SIGHUP signal to the snort-process, which makes snort reload its configuration, and start using its new ruleset.

6.7 Graphical User Interface

The Graphical User Interface (GUI) is a template- and AJAX-based[8] web application which uses Django's template system to implement the View part of the MVC-principle**??**. Each webpage is a template populated through Python-code before it is delivered to the Web server and ultimately the requesting Web browser.

Additional data that needs to be sent to a Web page, e.g. the user interacts with the GUI, is requested through JavaScript[9] and AJAX-calls and then gets added to the already loaded DOM[10] in the web browser.

6.7.1 Structure

The pages of the GUI is organized in the following structure, which is very similar to the way BHTB organized it's front end.

6.7.2 Design

The GUI is designed with simplicity and minimalism in mind, which is achieved partially by using Bootstrap that has a lot of default no-nonsense design elements. But we also opted for a lot of primary colors and simple lines to make it easy on the eyes.

[8] AJAX is web development technique that allows you to send asynchronous requests to a web server.
[9] A programming language commonly used for client-side scripts.
[10] The webpage content or elements.

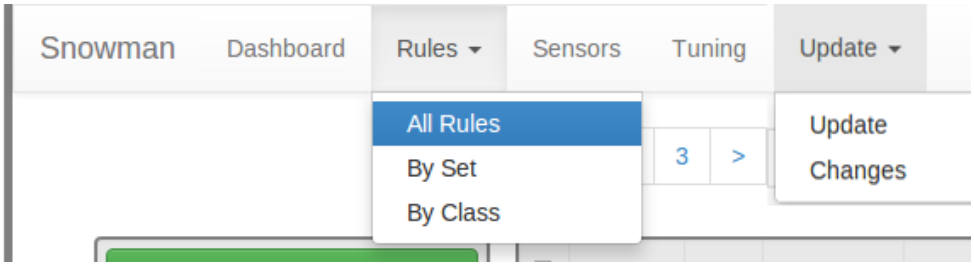


Figure 14: GUI Structure

6.7.3 Overview of central GUI features

Content Prefetching

One of the main features of our GUI design is our usage of prefetching to speed up loading. Instead of loading absolutely everything into the web browsers DOM, like our predecessor BHTB did, we instead initially only load the first page or page elements. This significantly reduces loading times.

We then silently in the background load in any element the user might want to see next, through AJAX calls. When the user then interacts with the GUI, the next elements are shown to the user. The next-next elements are also loaded through AJAX calls so that they are ready.

This ensures that the user always has immidiate response from any interaction done with the GUI, while keeping loading times to a bare minimum.

Rules page

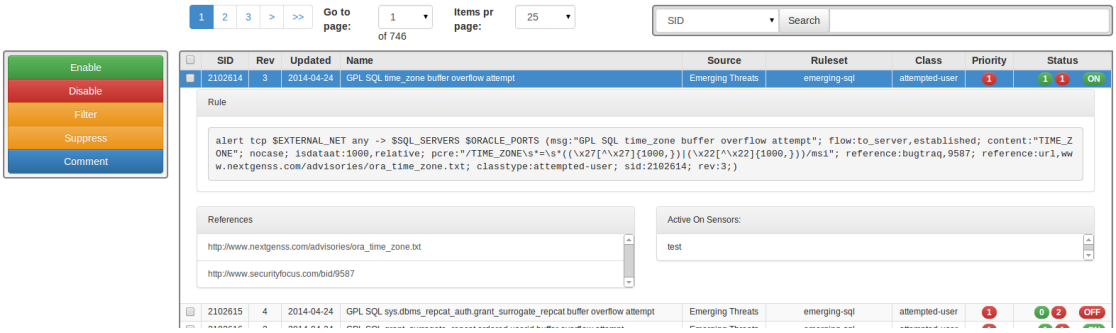


Figure 15: Rules page

This page serves as the complete list of all rules currently residing in the central database. Because the central database can literally contain hundreds of thousands of rules, this list is paginated. Pages are loaded using AJAX calls to prefetch them as needed.

Each row in the list can be clicked on, which expands an area containing additional information about the rule in question.

We anticipate that the main use of this page is to utilize the search function to find a specific rule or rules and then either retrieve information about them or otherwise manipulate them.

Rules can be tuned by checking the checkbox associated with the rule in question and

then pressing the button that represent the desired function in the box to the left.

Rulesets page

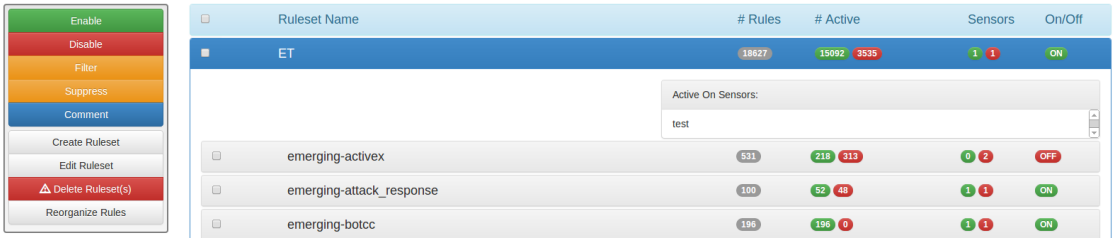


Figure 16: Rulesets page

The rulesets page consists of a list of all the rulesets stored in the central database. Each row contains meta information about the ruleset, such as the number of rules in the ruleset and so on.

A special feature of the ruleset page is that since Snowman allows you to structure rulesets in an hierarcical tree, the ruleset list will initially only show the tree roots. If the ruleset has child rulesets associated with it, those will be listed once you click on the ruleset and open up the additional information area.

The list controls also contain functionality for creating, editing, deleting and reorganizing the rulesets, as well as the ability to turn them on or off either globally or on a per sensor basis.

We also have a very similar page for the sensor list, which also allows you to arrange sensors in a hierarchical tree. Because it is virtually the same, it is not detailed in the report.

Updates and changes

One of the big requests from the client when it came to improvements of features of BHTB was the need to have more feedback over the update process, as BHTB only told you about the status after it was done, leaving the user in an information-limbo.

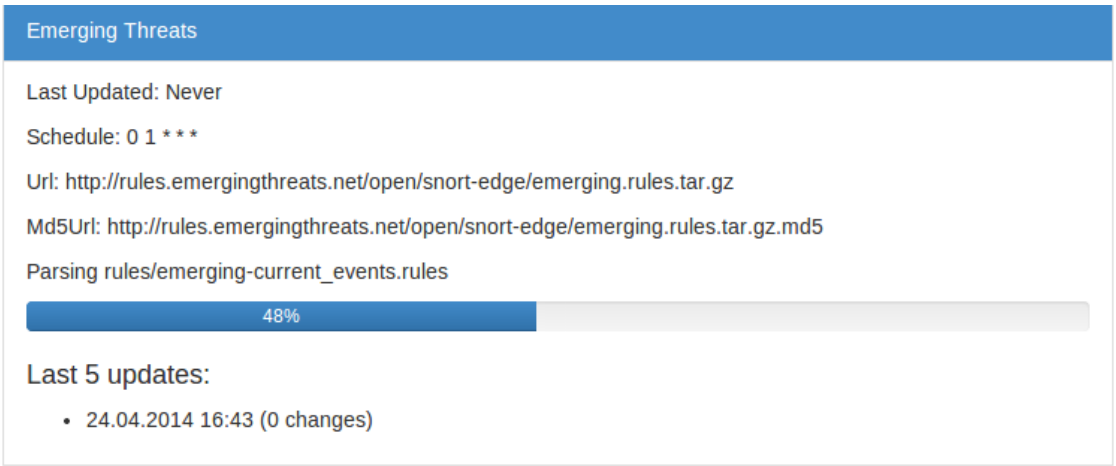


Figure 17: Update progressbar

To solve this, we added a continually updating progressbar which tells the user what the system is currently doing and giving a rough estimate of the percentage of completion.

Another requested feature was the ability to see what was changed during an update. We provide a list for this which contains each update and it's various changes.

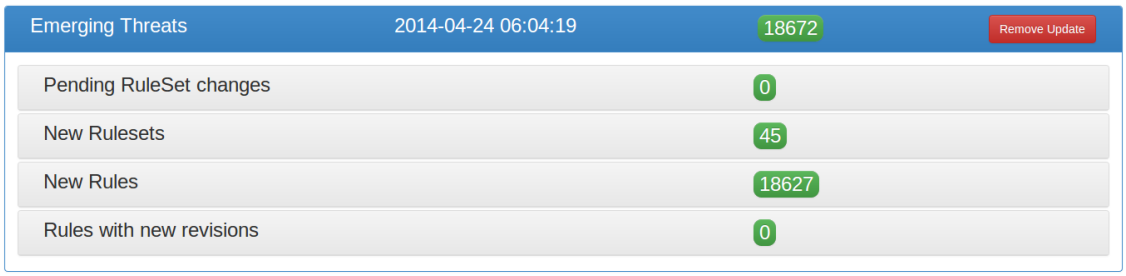


Figure 18: Update changes

The list details any new rulesets, rules or rule revision an update might have brough with it, as well as a list of rules that have been moved from one ruleset to another. Once the user has finished checking out the changes from one update, the user can remove the update from the list. This makes the list a backlog tool, rather than a perpetually long list or some sort of time limited list (say, updates in the last 7 days or something similar). This gives the user a good overview of what has changed since the last time the user checked the list.

Form validation and feedback

One challenge that always face web frontends are forms, as they are a means to alter data in the database and thus become a security challenge. So when you utilize forms, it is important to protect the database against both malicious attackers and user errors.

There are three basic steps to implement protection against these security issues, which is to (1) validate the data provided by the form, (2) make sure the user can't input something it shouldn't and (3) to never let any raw strings or values from the form be directly inserted into a database query.

The third protection is perhaps the most important as it leaves you vulnerable to SQL injection attacks, which are attacks where SQL code is input into the form, letting the attacker carry out his/her own queries, leading to total database compromise. This problem is solved through what is called prepared statements, which makes sure a query doesn't do anything it's not supposed to before the query is actually carried out on the database. This protection is implemented in our system through Django's internal protection mechanisms.

The first and second protection is something we have implemented ourselves at both the frontend and the backend. We make sure that the user can never input invalid values into any form by having Javascript validate the input. So if a user inputs characters where only numbers are allowed, the user will be notified immediately and not allowed to submit the form until it is corrected. And this is done before anything is ever send from the browser to the server.

But if the user has input all valid values, we still validate all the data at the backend

before we store anything in the database. This means checking that any referenced object ID actually exists or making sure the provided IP address is actually valid (within the 0.0.0.0/0 - 255.255.255.255/32 range). If anything is out of place, a message is sent back to the user about it.

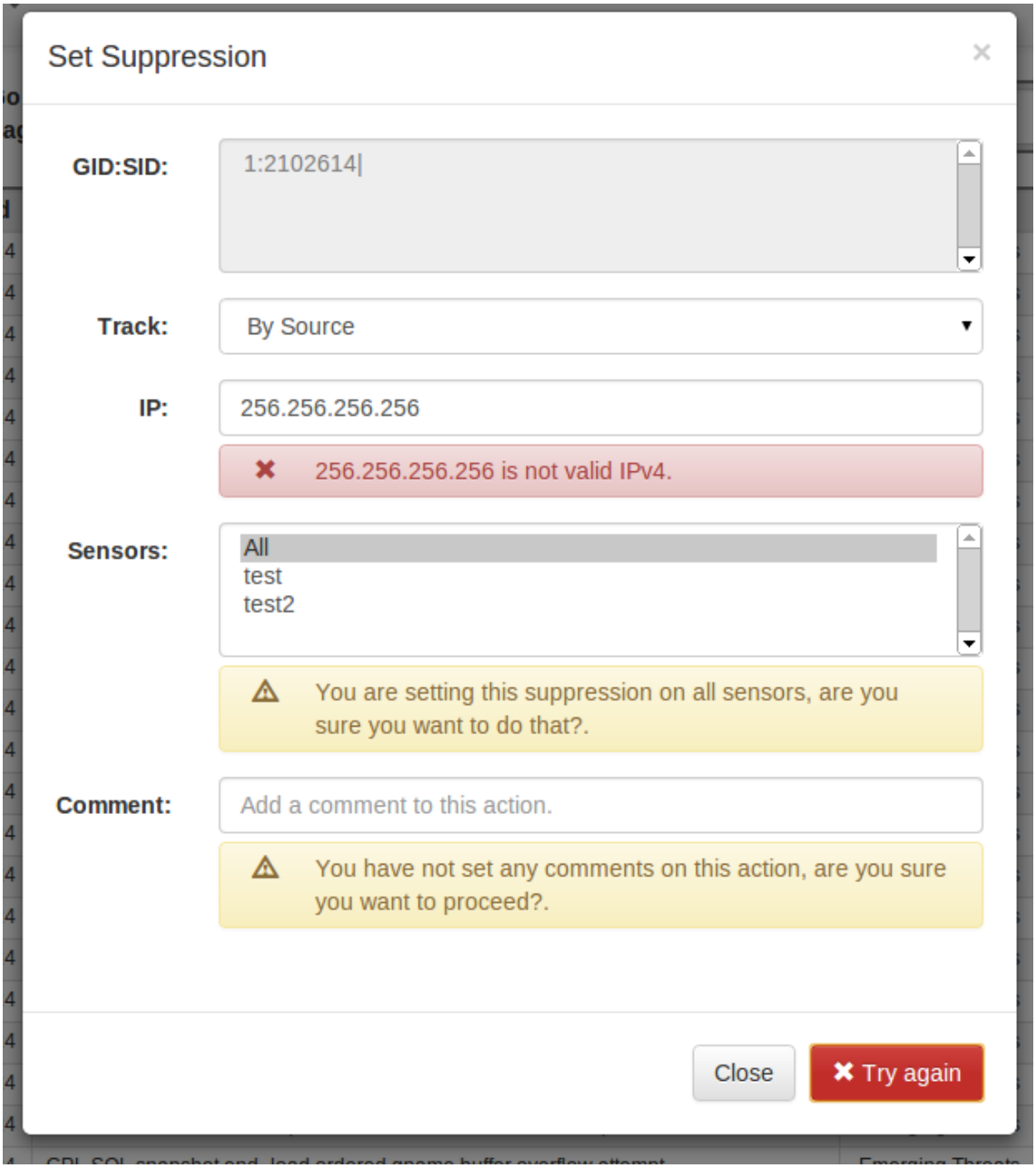


Figure 19: Form validation

6.8 Logging

Complex software needs to log what is happening, to simplify the understanding of what happend if things doesnt work as expected. It is also needed to document what is done,

in case someone would need to see if an update ran or simmilar. As Snowman is slightly more than a couple of small scripts, we need to think about logging of the internal workings.

For the logging of internal events in both the Snowman-client, and the Snowman-server, we decided to use the standard python logging libraries. These library allows us to log events with a certain severity inside our code, depending on how important these events are. The logger would then log from which modules the events appear, the time an event happens, and a message describing the event. Debug, Info, Warning, Error and Critical is the default logging-severities, sorted in an ascending order.

The logger can then get assigned handlers, which decides where to store the logged events. Snowman is by default using one handler, writing events to a file. The configurationfile for Snowman decides where this file is stored, and which severeties is stored. Under normal operation, it is recommended to not log the debug-messages, as they should not be needed, and therfore only use diskspace.

The python logging facilities are configurable, and flexible. If future needs would require some events to be stored in a database, or send an sms for that matter, the only thing that would be needed to be done is to add a new handler.

6.9 Configuration files

Snowman's configuration files exist to let users

The configuration files originated from a need to store database

some options

for example configuration files see appendix.

6.9.1 Server configuration file

set the number of revisions to store. Snowman will automatically delete the oldest revision when the revision count for one rule exceeds this number. The default value is 2, which is just enough to let users roll back to the previous working revision after an update. Users can also specify whether incoming revisions should automatically become active (i.e. go right into production) or remain inactive until manually activated.

There is a possibility that a rule will be moved from one rule set to another by a source, for example the ... rules contain a "deleted" set which contains rules that were previously part of other rule sets but is now considered obsolete. Users can also choose to re-arrange rules across rule sets, which may create conflict during updates. In the server configuration file, users can specify that rules should *a*) always be moved when an update contains the rule within a different rule set; *b*) moved unless a user has previously moved the rule; or *c*) not moved at all.

6.9.2 Client configuration file

Every client in the system requires its own configuration file. In this file, the credentials of the sensor on which the client operates are stored, along with database and logging options and paths to required directories and files. Eigil: please describe purpose behind rules per request.

During the design of Snowman the solution to some issues was not immediately clear. As an example, consider the automatic update of new rule revisions. Some users would like to have the new revisions put into production, but others might want to keep the new revisions inactive until they are considered good and ready.

It was decided that these types of issues should be specified in a configuration file.

For these types of issues, where the project group remained inconclusive

not clear whether it would be best to do it

(e.g. should new rules always assume a deactivated state or the opposite). These types of issues

Describe the purpose and structure of the config file.

Some update-packages may contain source code files or modules which are of no interest for Snowman.

Users can choose between using a whitelist (parse only whitelisted files) or a blacklist (parse all but the blacklisted files)

6.10 Documentation

How the software is documented.

6.11 Deployment

The deploying of snowman is a two-stage approach. First, the central server needs to be installed and configured first. After the server have beed verified to be working as intended, the Snowman-clients can be installed alongside of Snorton the sensor-hardware.

Both the server and the client is delivered packaged in debian based installation files. These installation-files will make sure to install the Snowman software, in addition to make sure that the required dependencies also is present on the machine.

However, there are one exception to the dependency solving. The server is delivered in both a debian-flavour in addition to an ubuntu-flavour. The reason for this is that some of the python-libraries used is currently not in debian-stable, and for debian it is required to install them via the python package-source, pip. After the installation, we therefore provide some utilities as a double-check, to be able to verify that all the required tools are present.

As all the Snowman software is delivered in packages, automation of the installation can be achieved trough the use of standard tools like aptitude, and configuration tools like puppet and cfengine.

6.11.1 Snowman server

To deploy the snowman-server, it is wise to do a couple of considerations first. Snowmanuses external software for web-server and database, and because Snowmanis a quite flexible piece of software, it is possible to change which technology is used, with a simple configuration-change.

The web-interface is powered by a python wsgi gateway, and thus is very flexible to which server is serving it. The only requirement would be that the webserver is supporting wsgi. As default, configurations is supplied that uses apache2.

For the database access, as the Django database ORM is used, several technologies can easily be used. Sqlite, Mysql (and thus MariaDB), PostgresSQL and Oracle is all supported. Sqlite might work for small installations, where simplicity is prefferred over performance. The only requirement for a sqlite database would be access to write a file in the filesystem. If performance however is the requirement, a dedicated RDMS would be prefferable. A standard and free alternative here is MySQL.

Installation

When the database-technology is selected, and the webserver is available, the installation of the Snowman package can be done. The package is delivered in two versions; one for debian and one for ubuntu. Both would work on both systems, as the only difference is the dependency-list. In the debian-version, a couple of dependencies is ommitted, as they are not in a new enough version in the debian-stable repositories. These dependencies needs to be installed manually using python pip:

- django (version >= 1.6)
- ??

Configuration

After the installation, a couple of configuration-steps remains. First, in the file "/usr/share/snowman/srm/settings.py" the following variables needs to be updated:

- SECRET_KEY: Create a random string, and insert here. It is used for signing messages, and make sure no tampering happens in the clients of the webinterface.
- ALLOWED_HOSTS: An array of hostnames used by the server. If the webside is requested with any other hostnames than those listed here, an error-page will be displayed instad of the webinterface.
- DATABASES: A dictionary which contains the database-settings. For more information on how to configure this settings, consult the django documentation [11]

The rest of Snowman's configuration is to be found in "/etc/snowman/snowman.cfg". All the configuration-parametres is well documented in the configuration-file, and they affect different parametres of the internal workings of snowman, and the snowman daemon.

To configure the webserver, an apache configurationfile is supplied in "/etc/apache2/conf-available/snowman.conf". If this is to be used, a symlink to this file can be created in "/etc/apache2/conf-enabled/", and apache restarted.

When the configuration-changes is complete, the command "snowman-initialize" should be used, to generate the database-schema, and intialize the database with some initial data. The output of this command would also signal if the database-settings is correct, as it cannot create the initial data without access to the database.

Verification

When Snowman is configured, it is time to verify that the configuration is valid. There are 5 things that needs to be verified:

- Database availability.
- Access to logging locations
- Access to scratch-location on the filesystem.
- Availability of the webinterface
- Availability of the socket used by clients to download updates.

[11] https://docs.djangoproject.com/en/1.6/ref/settings/#databases

The command “snowman-test” is supplied to have an automated way to test all these parametres. If everything is ok, a message is stating so, and the program returns an 0. If any of the above bulletpoints fails their corresponding test, A message is dispayed, and the test returns a 1.

6.11.2 Snowman client

The deployment of the snowman-client is a simple three-step approach:

1. Install the snowman-client package
2. Update “/etc/snowman/snowman-client.config” with database-settings, address for the central server, sensor name and sensor secret.
3. Add the sensor to the Snowman-server, for example trough the webinterface.

To be able to verify that everything is as it should be, a tool is provided to test that the client is properly installed and configured. The command “snowmanclient-test” is verifying all these points:

- The required libraries is present.
- The configuration specifies a database that is accessible.
- The location for creating rule-files is writeable.
- A contact to the central server is able to be established.

snowmanclientd

There are supplied a daemon together with the Snowman-client, with the name “snowmanclientd”. It is responsible for letting the central server make contact to the client. It is set up to automatic start on boot, but it is also possible to control via the ordinary linux command “service”: “service snowmanclientd start|stop”

7 SOFTWARE TESTING

Her beskrives hvordan opplegget rundt testing av applikasjonen har vært. Dette omfatter planlegging, valg av teststrategier, gjennomføring av tester og beskrivelser av sentrale feil funnet under testing, samt måten feil ble rettet opp på.

7.1 Strategy

The overall strategy of the project when it comes to testing has, first and foremost, been to have regular meetings from the client where we demonstrate live demo's of the software. This has lead us to either confirm things as feature complete or created a list of things to either fix or improve, along the way.

We have also not had any specific regimes when it comes to bugfixing, we have mostly just handled them when they have been discovered them.

You could describe our software testing strategy as ad-hoc based, as is quite common in agile projects. Our main focus has been on the performance testing.

Unit testing has not been used extensively in this project since the development team has limited knowledge and experience with this type of testing. It has been used, however, to test that data from an update is correctly inserted into the database. Source code listings of all unit tests in this project can be found in appendix.

7.2 Description of Problems

7.2.1 Rule insertion

Snowman extracts various types of data from the rulestring and inserts this into the database in different tables. Since an update can contain thousands of rules it proved very difficult to effectively verify that all rules were correctly inserted into the database. To address this, a test suite was set up executing a dummy update with an example rulestring containing all possible options.

```
def test_updateRule(self):
  # Insert the rule
  self.update.updateRule(self.rulestring, "example.rules")

  try:
    # Verify that all related objects exist
    rule = Rule.objects.get(SID=1, active=True, priority=10)
    generator = rule.generator
    ...
  except Rule.DoesNotExist:
    self.fail("Rule does not exist")
  except Generator.DoesNotExist:
    self.fail("Generator does not exist")
  except ...

  self.assertTrue(rule.active==True)
  self.assertTrue(int(rule.priority)==10)
```

```
18    self.assertTrue(generator.GID==1)
```

These tests were also very useful during the refactoring of the Update module.

7.2.2 List Pagination

One problem that has plagued the software throughout the development has been the pagination functionality in the GUI. Due to it using AJAX to fetch other pages in the background, if you changed pages so fast that the system couldn't catch up, it would just end up never loading anything due to AJAX's asyncronous nature[1]. This has been remedied by using some logic to check for outstanding AJAX calls but a better solution would be to implement some other mechanism or framework for pagination. The remedy has also put some extra latency on the rule list as waiting for the AJAX calls might take some time.

7.2.3 The Ever Changing Number of Changes

One smaller bug that popped up was on the Changes page where the number of new rulesets, rules and rule revisions there has been in the latest updates were displayed. Initially, only the rule revision was tied to the Update object and we then traversed from the revision object to the rule object and count how many revisions that rule had, in order to determine if the rule was new[2]. But using this method led to a undesired outcome when a "new" rule gets a new revision in later sequent update. As the count for revisions is done real time, this rule will suddenly not be listed as new anymore in the old update.

This problem was solved by tying all relevant new objects to the Update object and then just list those relations. The downside of this is of course a lot more relations to store in the database, but the Changes view was one of the more important features so a camel had to be swallowed.

[1] The code does not wait for AJAX to finish before moving on.
[2] A shiny new rule will always only have one revision.

8 PERFORMANCE ANALYSIS

8.1 Theoretical Analysis

The purpose of the theoretical performance analysis is to examine the most important features of Snowman and similar software, to form opinions on the expected performance of each software. The theoretical analysis is used to create concrete benchmarking tests presented in section 8.2. The software analysed is BHTB, Snortmanager and two versions of Snowman.

8.1.1 Effectiveness of Update Processing

As described in chapter **??**, Snowman extracts several rule options from the rulestring during parsing. These options will either create or update objects connected to the rule, such as rule set, rule message and classtype. These logical relations add to the complexity of the database transactions during the update process, as multiple database tables are involved. BHTB and Snortmanager, on the other hand, write rule data to a single table. It is therefore expected that BHTB and Snortmanager will generally exhibit a more effective update processing.

Incremental updates

One of the expected strengths of Snowman lies in processing incremental updates, i.e. updates containing only a small number of changes. It is significant to gauge performance for incremenal updates, as most real-life updates usually contain between 10 and 100 changes. Ten updates from 20.03.2014 to 17.04.2014 contained on average 36.8 new and 53.9 updated rules[1]. Snowman will effectively handle these updates, as it stores the checksum of the files in the update. Storing the checksum for every individual file[2] is unique for Snowman. It allows Snowman to completely ignore files without changes, which means that only the files with rule changes are parsed. This is particularly useful as the current official Snort rules contain over 100 files[3]. It is therefore expected that Snowman will effectively process updates that contain relatively few changes.

8.1.2 Effectiveness of Rule Distribution

The rule distribution, or synchronization, is a feature of Snowman and BHTB where rules are distributed from the server to the sensors. Since Snowman and BHTB use different strategies for distribution, Snowman uses a local client database as described in chapter 5.1.1 while BHTB pushes all files out to the sensor at each synchronization, it is previsioned that Snowman will perform better because of the fact that only transfers over new changes rather than everything, every time. Also, since BHTB distributes everything, every time, it is expected to have a flat performance curve, since the timing will be dependent on the bandwidth rather than processing power.

Snowman's server-client connection is more complex than the one of BHTB expect a longer processing time for setting up the connection.

[1] Source: http://www.snort.org/vrt/advisories
[2] Checksum is stored only for files that are parsed.
[3] As of May 2014

BHTB operates with a more crude way of distributing rules to sensors.
Snowman will only synchronize changes

8.1.3 GUI Latency

A major complaint from the client was that they experienced a large latency in BHTB's web component, that they had to wait long periods for it to load data. We've attempted to remedy this problem in Snowman by not loading absolutely everything into the GUI every time, like BHTB does. It is expected that this will reduce the latency in the GUI, as we would be processing and displaying only 100/28000 rules as opposed to all 28000 rules.

8.1.4 Linearity in Performance

We're expecting all of the software to have a linear performance curve, or better (downwards tendency), as an upwards pointing performance curve is a sure sign that something has been programmed very wrong. If performing one task takes 1 second, performing 10 tasks should take 10 seconds or less.

8.2 Practical analysis

Since Snowman is similar to BHTB and Snortmanager, it has been important to benchmark their performance in various formal test cases in order to highlight the strength and weaknesses in each software solution. The purpose of the tests is to reach final conclusions on key questions regarding the performance of Snowman, and to point out where the difference between the three software packages lies.

To get a picture of how the software performs on different systems, it was tested on three architectures (table 5). To minimize error, each test was run five times where the mean value of the runs is considered the valid test result. All tests can be reproduced with the test procedure documentation in appendix.

8.2.1 Definitions

The testing is split up into *test suites*, which contain a *problem* in the form of a question. The problem forms a clear description of what is to be tested, and how results should be evaluated. A test suite also contains one or more *test cases* which exist to address the problem. Test cases represent individual tests, and describe the input data and constraints. Test cases are carried out on the different systems, and results from each test case are reffered to as the *test results*.

8.2.2 Environment

All tests are carried out on three systems, named *S1*, *S2* and *S3* (table 5). These systems differ in processing power, available memory and storage technology, and the results will indicate what impact these factors have for software performance. The performance testing in this report is also prone to error, as tests might get disturbed by other background processes etc. Testing on three systems gives results with a higher confidence, as results are checked aginst each other for inconsistencies. When an inconsistency in results is found, the test is run again to see if the inconsistency prevails.

8.2.3 Test suites

Even though Snowman, BHTB and Snortmanager share the same goal of being a rule management system, it is not immediately obvious how they can be tested against each

System	Processor type	Memory	Storage
S1	Intel® Core™ 2 Solo, 1.4GHz	4GB RAM	5K RPM HDD
S2	Intel® Core™ i7, 6x3.2GHz	12GB RAM	10K RPM HDD
S3	Intel® Core™ i5, 4x3.2GHz	16GB RAM	SSD

Note: See appendix for full details.

Table 5: Test systems.

other to compare their performance. The first step in the performance testing has been to identify the areas where the three software packages share, to a certain extent[4], the same functionality. In the test planning, considerations have been made to ensure that the tests hold an acceptable degree of fairness (i.e. tests are designed so that none of the software enjoys any improper advantages or disatvantages), and that the tests are pertinent to the daily operation of the software and the Snowman requirement specification.

Snowman, BHTB and Snortmanager all have the functionality to download and process an update from an external source. The first group of test suites presents different test cases to benchmark how each software performs in update processing.

8.2.4 Update processing tests

The purpose of the tests in this section is first and foremost to test how quickly the software can process an update, and if the processing time is linear to the update size. It is interesting to test for linearity as this may help point out programming errors. To get a complete picture of the performance, the tests are set up with different constraints such as update size, number of files and number of changes from one update to the next.

Tests numbered 1-1 through 1-4 use an artificially generated update package where all rules contain the same data[5].

Test 1-1

Problem: How fast can the software process an update?

The purpose of this question is to determine the average processing time per rule. Results are ranked from short to long processing time, where shorter is better.

Test cases:

To address this problem, five test cases were introduced. The tests contained an artificially generated update with 1, 10, 100, 1000 and 10000 rules, respectively. Rules were split into files with ten each. Each test case represents one update, and the names denote the number of rules in the following structure: *SYN-number of rules*. The test is also carried out with a realistic update package, *SF-28064*.

[4] Snowman, BHTB and Snortmanager have been built to tackle many of the same challenges, however, the actual implementations may differ severly. For an extended walk-through of differences in implementation, refer to chapter 2, and chapter 10.

[5] Only signature IDs are different. The rules contain a message and a classtype, see appendix for full description.

Test case	In test	Description
SYN-1	Test 1-1, 1-2	Blank database.
SYN-10		
SYN-100		
SYN-1000		
SYN-10000		
SYN-1000-B	Test 1-3	All rules in one file.
SYN-1000-C	Test 1-4	Pre-populated database. No changes in update.
SYN-1000-D	Test 1-5	Pre-populated database. Revision change for all rules
SF-28064	Test 1-1	Blank database.
SF-28064-B	Test 1-6	Pre-populated database. Update has one rule change.

Note: Rules are split into files with ten rules per file unless other is specified.

Table 6: Test cases in update processing.

Test 1-2

Problem: How strong is the linear correlation between processing time and the size of an update?

This question rises from the requirements specification of Snowman (chapter 3), where it is stated that Snowman shall exhibit a linear increase in processing time in correlation to the number of rules in the update (where the number of rules depicts the update size). Results are ranked from strong correlation to no correlation, where stronger is better.

Test cases:

This test uses the same test cases as Test 1-1.

Test 1-3

Problem: Does raising the ratio between files and rules increase processing time?

Updates can come with the rules either gathered in one file, or distributed over several files. This question is asked to examine the difference in processing time for these two scenarios. Results are ranked from no difference to big difference, where less difference is better.

Test cases:

This test compares the *SYN-1000* test case from Test 1 with a test case with the same number of rules in *one* file (*SYN-1000-B*).

Test 1-4

Problem: Is the processing time shorter when the update contains no new rules?

This is a test for update processing when the update is not new (i.e. it has been completely processed by the system before). To minimize system load, it is essential that already processed updates are (in practice, they should be neglected) by the system. Results are ranked from shortest to longest processing time, where shorter is better.

Test cases:

This is a compound test where a test case is run two times in succession. The *SYN-1000* test case from Test 1 is run first to populate the database, before it is run again (*SYN-1000-C*).

Test 1-5

Problem: Are the processing times for two updates identical when only revision numbers are changed between the updates?

This question measures how one small change in all rules affects procesing time. This test runs the *SYN-1000* update to populate the database, and then the *SYN-1000-D*, which contains a revision number change for all rules. Processing time is expected to be faster for the second update, since the database will already be populated with the majority of the data. Results from this test will be ranked by the difference in processing time between the first and the second updates, where bigger difference is better.

Test 1-6

Problem: Are the processing times for two updates identical when only one rule has changed between the updates?

This question seeks to determine how the software handles an update when there is only one change from the previous update. This is an important factor to consider, since an update usually contains only a small amount of changes from previous updates. Results are ranked from short processing time for second update to no difference in processing time, where shorter time is better.

Test cases:

This test suite contains two test cases with an update from a real source. The *SF28061* is run first to populate the database before *SF28061-B* is run, which contains one change (one of the rules has an updated revision number).

8.2.5 Rule distribution tests

8.2.6 Interface loading tests

These tests will evaluate the speed of the user interface. Listing rules and rule sets. Uses timer in Google Chrome. Both tests use the SYN-10000 package. The reason for using this is..

Test 3-1

Problem: How quickly does the user interface list rules?

Test cases:

This test uses one test case, called *list rules*, where 100 rules are listed in the user interface.

Test 3-2

Problem: How quickly does the user interface list rule sets?

Test cases:

This test uses one test case, called *list rule sets*, where all rule sets are listed in the user interface.

8.2.7 Abandoned tests

Not conducted due to time limitations. real-world tests. Would have been interesting to run distribution test to multiple sensors to verify parallellization. Tests with more realistic

rules. Testing of CPU usage.

8.3 Optimization

Test results for update processing (see chapter 9) showed that Snowman version 0.4 was very slow compared to the other software. The reason for this was examined, and found to be a large amount of database calls spread out in the code. Where BHTB and Snortmanager used bulk insertion into the database, Snowman 0.4 inserted objects as they were found and requested objects as they were needed in the code. To solve this, the project group split the methods of `Update` into two new classes: `Parser` and `Updater`, where the first would only parse the incoming data and send this to the latter which gathered all objects and wrote these in bulk when the parsing was complete. As can be seen in chapter 9, this had a huge impact on performance.

```
1  from core.models import RuleClass
2
3  def updateClassification(type, description, priority):
4    try:
5      # Update existing classification
6      ruleclass = RuleClass.objects.get(classtype=type)
7      ruleclass.description = description
8      ruleclass.priority = priority
9      ruleclass.save()
10   except RuleClass.DoesNotExist:
11     # Add new classification
12     ruleclass = RuleClass.objects.create(classtype=type, ...)
```

Listing 8.1: Updating a classification in Snowman 0.4

9 RESULTS OF PERFORMANCE TESTING

This chapter presents the results from the performance testing.

Mean times calculated from five sequential test results. See appendix for a complete listing of results.

9.1 Update processing

Test case	System	Snowman v 0.4	Snowman v 0.5	Snortmanager	BHTB
SYN-1	S1	1058.179	1415.484	228.617	125.236
	S2	622.640	804.681	93.403	65.281
	S3	151.946	196.160	36.960	18.175
SYN-10	S1	2289.518	1427.712	265.275	148.323
	S2	1668.142	808.218	107.554	72.025
	S3	327.655	195.159	37.569	19.711
SYN-100	S1	17519.385	1568.224	1251.275	185.970
	S2	12197.192	865.496	139.005	84.913
	S3	2356.610	343.033	159.113	25.024
SYN-1000	S1	167645.267	2568.235	9470.460	568.333
	S2	114617.561	1098.159	460.394	158.351
	S3	22830.358	1823.785	1360.276	60.313
SYN-10000	S1	1795175.128	14179.095	92428.555	3606.043
	S2	1162468.835	5761.811	5105.158	910.172
	S3	228038.066	16804.950	13523.887	440.334
SYN-1000-B	S1	147170.285	2508.530	1836.396	445.476
	S2	113901.478	1245.906	496.204	157.817
	S3	20367.838	404.022	228.044	49.908
SYN-1000-C	S1	296.272	309.576	1763.480	7.816
	S2	186.215	169.476	829.230	4.543
	S3	56.152	54.824	205.956	1.310
SYN-1000-D	S1	1371.722	1431.436	64100.000	563.682
	S2	9682.763	748.258	31999.710	85.289
	S3	16998.276	2022.763	10716.689	83.275
SF-28064	S1	9175886.000	63961.545	43391.700	5638.424
	S2	3817584.070	26875.246	21127.828	1715.521
	S3	1267483.848	12522.786	8039.046	1017.561
SF-28064-B	S1	28200.953	9239.670	239878.897	5606.518
	S2	22673.225	5489.009	80009.077	2396.780
	S3	5260.553	1707.588	63626.550	1004.424

Table 7: Test results for update processing.

Test case	System	Snowman v 0.5	BHTB
SYN-1	S1	1645.639	1501.465
	S2	1503.815	1467.612
	S3	371.537	3.300
SYN-10	S1	1828.779	1489.364
	S2	1752.954	1472.808
	S3	453.362	3.462
SYN-100	S1	3975.408	1489.645
	S2	4036.130	1473.355
	S3	1094.713	4.492
SYN-1000	S1	24921.176	1771.004
	S2	25374.218	1503.511
	S3	7759.566	15.132
SYN-10000	S1	239641.965	1839.841
	S2	251607.202	1593.889
	S3	16862.160	121.677
SYN-1000-B	S1	24916.964	1522.719
	S2	25228.116	1481.411
	S3	7252.062	4.156
SYN-1000-C	S1	4265.931	1775.423
	S2	4297.144	N/A
	S3	2278.760	1097.711
SYN-1000-D	S1	6523.340	1775.664
	S2	6544.584	1513.944
	S3	11642.046	1064.944
SF-28064	S1	112509.497	1758.258
	S2	105555.617	1584.972
	S3	40004.218	1062.806
SF-28064-B	S1	26362.894	1754.601
	S2	23033.204	1584.696
	S3	8839.964	1117.693

Table 8: Test results for sensor synchronization.

Test case	System	Snowman v 0.5	BHTB
List rules	S1	2508	924.561
	S3	1045.950	657.408
List rule sets	S1	11306	39432
	S3	4676	10524

Table 9: Test results for interface performance.

10 ANALYSIS OF PERFORMANCE TESTING RESULTS

10.1 Effectiveness of update processing

This section discusses the most important results from the update processing tests. Table 10 displays the average processing time per rule calculated from the synthetic update containing 10000 rules and the realistic update containing 28064 rules. As can be seen, BHTB is significantly faster than the two other across all systems. It came as a big surprise that BHTB performed so well in the update tests, as our client had listed poor update performance as one of its major defects. The results are probably best explained by the very simple nature of BHTB's data structure. Also note that these tests might not reflect the reality that our client faces, as they are run in a minimalistic environment without any sensors or other database-objects that might affect BHTB's performance in a real environment. Snortmanager also performed well in the tests, which is expected as it has a very similar data structure to BHTB in that it is very simple.

Test	System	Snowman 05	Snortmanager	BHTB
SYN-10000	S1	1.4179	9.2429	0.3606
	S2	0.5762	0.5105	0.0919
	S3	1.6805	1.3524	0.0440
SF-28064	S1	6.3962	4.3392	0.5638
	S2	2.6875	2.1128	0.1716
	S3	1.2523	0.8039	0.1018

Note: All times in milliseconds.

Table 10: Average processing time per rule.

Snortmanager starts to struggle when more files are introduced into the tests. As can be seen in table 11, there is a big difference between tests *SYN-1000* and *SYN-1000-B*, where we have the same number of rules, but spread over more files. This problem does not seem to exist in either Snowman nor BHTB.

None the less, both Snortmanager and BHTB pummelled Snowman v0.4 into the ground in these tests, which led us to completely rework our update program workflow. Our problem was that for every rule, we were writing its data to the database and then moving on to the next rule. This caused a lot of busy-waiting for the database. You can see that the times accelerated a lot on the machine with the lower end harddrives, so the problem also manifests itself more when the harddrives can't keep up.

We proceeded to rewrite the code so that all the rules are parsed into memory first and then is committed into the database in one big query. As you can see in table **??** we improved Snowman's performance in v0.5 with as much as xx% in some tests.

While we still didn't fulfil the performance goal, considering the difference in complexity and structure, we feel the difference is acceptable. The results in this test is a very

Test	System	Snowman 05	Snortmanager	BHTB
SYN-1000	S1	2568.235	**9470.460**	568.333
SYN-1000-B	S1	2508.530	1836.396	445.476
SYN-1000	S2	1098.159	460.394	158.351
SYN-1000-B	S2	1245.906	496.204	157.817
SYN-1000	S3	**1823.785**	**1360.276**	60.313
SYN-1000-B	S3	404.022	228.044	49.908

Note: All times in milliseconds.

Table 11: Comparison of processing time for many files vs one file.

good indicator that performance testing during development is a very useful thing to a development project.

10.2 Effectiveness of rule distribution

As we expected, BHTB had a very flat performance curve in this test, but not exactly flat. There is a small increase in time spent when the number of rules go up, as you can see in table **??**, meaning there is some degree of processing time infuencing the results.

Still, considering the tests on S1 and S2 were carried out on the same network and sensor, and S3 were carried out on a completely different network and sensor, it is apparent that distribution from BHTB is mostly determined by how fast the data can be moved from the server to the sensor.

The same difference in testing environments is apparent in the Snowman tests. S1 and S2 have pretty much the same results while S3 probably benefits from it's different hardware and network setup. This leads us to conclude that the performance of distribution is dictated by the bandwidth and by the hardware of the sensor, and not so much the hardware of the central server.

However, there does not seem to be a pure performance gain over BHTB as we expected by going for a distributed model with a change based distribution protocol. But we are seeing a significant reduction in time spent distributing when there are only updates and not a massive batch of rules. While BHTB uses roughly the same time on tests SF-28064-A and SF-28064-B, we use about 80% less time on test B than on test A.

This gives merit to the model we picked for our distribution architecture and with a little more performance optimizations, the overall times might improve further as well.

10.3 GUI latency

These tests probably entails the most grim part of our performance results as we seeminly have failed to produce an improvement over BHTB when it comes to displaying rules, something that does not make much sense if you do the math. We are only fetching and displaying a fraction of the number of rules that BHTB does[1], yet we spend about twice as long on it.

We have not spent much time investigating the cause of this, but we strongly suspect its a combination of our more complex data structure, coupled with our rather novice

[1] About 100 to 28000.

use of a complex template framework such as Django.

A small optimization was attempted by enabling a setting that lets Django prefetch related objects from the database and that did improve the results somewhat, but not enough to be noticable.

10.4 Linearity in performance

Table 12 shows the linear correlation between the test results for the synthetic updates with 1, 10, 100, 1000, and 10000 rules. The first dataset is the test results and the second dataset is $\{1, 10, 100, 1000, 10000\}$. As can be seen, the performance is close to linear for all software across all systems.

System	Snowman 05	Snortmanager	BHTB
S1	0.999945	0.999999	0.999695
S2	0.993996	0.999627	0.999931
S3	0.999999	0.999999	0.999983

Note: All times in milliseconds.

Table 12: Empirical linear correlation coefficients for update processing.

10.5 Conclusions

- Snowman kom dårligere ut - Fordi større data struktur - Optimaliseringen i update var god - Viser viktigheten av performance testing i prosjekter

11 CONCLUSIONS

11.1 Results

This project set out to develop, from the ground up, something that were to be better than a prototype already in use. And we have produced some that does indeed works and, in many cases, better than it's predecessor. We did not manage to build in all the features we outlined in our goals, but the features missing are also features not present in the predecessor BHTB.

We discovered through our performance tests that we did not reach our goals when it came to making Snowman more effective than BHTB, but this can be acredited to the more complex data structure which results in more data needing to be processed and moved to/from I/O devices.

Our performance tests in chapters 8 to 10 also revealed that actually conducting them in a software development project is a very good idea, as we discovered a big bottleneck in our Update module and managed to fix that and improve our performance significantly. Some aspects of Snowman are still not within acceptable performance times, but the optimization done on the Update module brought it into an acceptable range.

While we have not conducted tests on the matter, we believe Snowman does not have the same scalability problems that BHTB seems to have when adding more sensors to the system. This has to do with our distributed model, removing the bottleneck of being dependant on a central system for the distribution of rules and removing the bandwidth strain of distributing everything, every time.

We also created a GUI that does the same and more than what the GUI of BHTB does. It is not perfect by any stretch of the imagination, but it features more essential features such as form validation and user feedback.

The project also reveals that using frameworks such as Django to create an object abstraction on top of the database can really let you create a lot of functionality in a shorter amount of time, than if you had to work directly on the database.

And last but not least, the members of the project group has learned a great deal when it comes to software development projects, especially when it comes to working with bigger enterprise oriented frameworks such as Django, when it comes to time management and about IDS systems in general.

11.2 Future Development

At the start of the project we were ambitious with our project goals and the client had a long list of functionality they wanted implemented. While a large chunk of the functionality made it to our latest version, we had to sacrifice some due to time constraints.

Which were the functions we put in the "if time permitting" pile from the start, the ability to comment rules, the ability to write new custom rules and to have some sort of way to check for errors in the rules before they are distributed. There is, however, no problems adding these features in future work, as we've strived to make the program modularized.

To solve the functionality of checking for errors in rules, we discussed early on if something like Dumbpig[27] could be used. It gives feedback on rule syntax as well, so it could also be used to check rules that have been written by the user for mistakes or errors.

There are also more optimization work that could be done to speed things up, especially when it comes to fetching lists of rules. Rule distribution will also benefit from a similar optimization as done with rule updating.

The GUI could probably also benefit from more work, as it has a few quirks and odd bugs. It would probably be beneficial to find some other way of doing pagination, as the implementation in the current iteration is not optimal. There's also some cosmetic improvements to be done to tighten up the looks, there are a few places where theres too much clutter once you've navigated and opened too many things.

We also compiled an improvements list from our client on a few matters, which is available in appendix **??**.

11.3 Group Evaluation

11.3.1 Introduction

At the beginning of this project we all agreed on that we wanted to reach for that often unreachable top grade on this project. To accomplish this, we've worked countless hours to the best of our abilites and we've had virtually no issues or problems to sidetrack us as a group, so we've been able to fully focus on the task at hand.

The group has worked well as both a team and as individuals, drawing from each others strengths and expertise. We've created something that could very well either live on in service of the information security community or inspire others to further improve or recreate.

The work that has been done over the past few months is something the group can be proud of.

11.3.2 Organization

While we had a designated group leader, we never really got in a situation where we needed one. Most of our decisions or problems were solved as a group through discussion and a democratic approach and that worked well for our group.

It seems like we hit the nail pretty much dead on when it came to our project plan, which we have more or less followed throughout the entire project. The development cycle lengths and intermittant meetings has allowed us to be very flexible and agile in our development, as well as being more or less on schedule.

Spending some time planning everything before we started programming gave us a good head start and even saved us some time writing this report as we had a lot of its components already made. When we completed the initial planning phase, we held a "project decision meeting" with the client where we introduced the system models we had made so that the client could decide what they wanted or not. We felt this was a good idea, as we could then get some valuable feedback before we started programming.

Our approach to the development cycles was mostly as described in section 1.7.1, we conducted syncronizational meetings once or twice a week to chart out the course and keep up to date with what the others were doing. But for the most part, because we had devided the software into modules, we worked individually. We feet this was the way

to go, since we could all work comfortably at home in front of our favourite computer, being more productive that way.

When it came to the thesis, we set a good month and a half of our project time for it and we think it paid off, especially as we expanded it to include the performance testing part. We also devided the writing part into three draft cycles, so we could get valuable feedback from our mentor and client.

11.3.3 Work distribution

For the most part our work distribution has been a “free for all”, meaning the group members picked freely from what they wanted to work on during the development cycle, within the objectives we set for that cycle. This worked out fine for the most part, but we feel we probably could have been slightly more aggressive with how many features we finished within the cycles. This left with with a bigger workload towards the end of the development as opposed to the beginning.

11.3.4 What could have been done differently

As most bachelor thesis' write towards the end, “if only there was more time”. In hindsight, we probably could've conducted our performance testing earlier in the project, so that we could have more time to implement optimizations. We should've also had a more strict approach to what we were supposed to work on for the different development cycles, planning that out earlier as well.

11.3.5 Subjective views

Thomas Nyheim

Skriv om folelser og slikt

Eigil Obrestad

Skriv om folelser og slikt

Eirik Skogstad

Skriv om folelser og slikt

11.4 Conclusion

This was fun and stuff.

Bibliography

[1] St.Laurent, A. M. 2008. *Understanding Open Source and Free Software Licensing*. O'Reilly Media.

[2] Roesch, M. 1999. Snort - lightweight intrusion detection for networks. *Proceedings of LISA '99: 13th Systems Administration Conference*.

[3] Henriksen, D. O. Managing signatures for ids in a distributed environment - a study of a signature management system. Master's thesis, Gjøvik University College, 2012.

[4] Vaskinn, C. & Wikestad, K. Snortmanager. Bachelor's thesis, Gjøvik University College, 2012.

[5] April 2014. Django v1.6. `http://www.djangoproject.com/`.

[6] May 2014. Snorby. `http://snorby.org`.

[7] May 2014. Surikata. `http://suricata-ids.org`.

[8] Cohen, D., Lindvall, M., & Costa, P. 2003. Agile software development. *DACS SOAR Report*.

[9] Wikipedia. May 2014. Waterfall development method. `http://en.wikipedia.org/wiki/Waterfall_model`.

[10] Wikipedia. May 2014. Scrum development method. `http://en.wikipedia.org/wiki/Scrum_(software_development)`.

[11] May 2014. Tornado. `http://www.tornadoweb.org/`.

[12] May 2014. Sqlite homepage. `http://www.sqlite.org/)`.

[13] Roesch, M. et al. 2012. *Snort Users Manual 2.9.3*. The Snort Project.

[14] Woodside, M., Franks, G., & Petriu, D. C. 2007. The future of software performance engineering. *IEEE Future of Software Engineering '07*.

[15] Vokolos, F. I. & Weyuker, E. J. 1998. Performance testing of software systems. *WOSP '98: Proceedings of the 1st international workshop on Software and performance*.

[16] Denaro, G., Polini, A., & Emmerich, W. 2004. Early performance testing of distributed software applications. *WOSP '04: Proceedings of the 4th international workshop on Software and performance*.

[17] Myers, D. On the use of nand flash memory in high-performance relational databases. Master's thesis, Massachusetts Institute of Technology, 2008.

[18] Eeles, P. November 2005. Capturing architectural requirements. http://www.ibm.com/developerworks/rational/library/4706.html#N10073.

[19] February 2014. Gnu general public lisence v3. http://www.gnu.org/licenses/gpl.html.

[20] May 2014. Python v2.7.6 documentation. http://docs.python.org/2.

[21] May 2014. Should i use python 2 or python 3 for my development activity? http://wiki.python.org/moin/Python2orPython3.

[22] May 2014. Python 2.x vs 3.x use. http://wiki.python.org/moin/2.x-vs-3.x-survey.

[23] May 2014. Django documentation. http://media.readthedocs.org/pdf/django/1.6.x/django.pdf.

[24] May 2014. Pep 257: Docstring conventions. http://legacy.python.org/dev/peps/pep-0257.

[25] April 2014. jquery. http://jquery.com/.

[26] April 2014. Bootstrap. http://getbootstrap.com/.

[27] May 2014. Dumbpig homepage. http://leonward.wordpress.com/dumbpig).